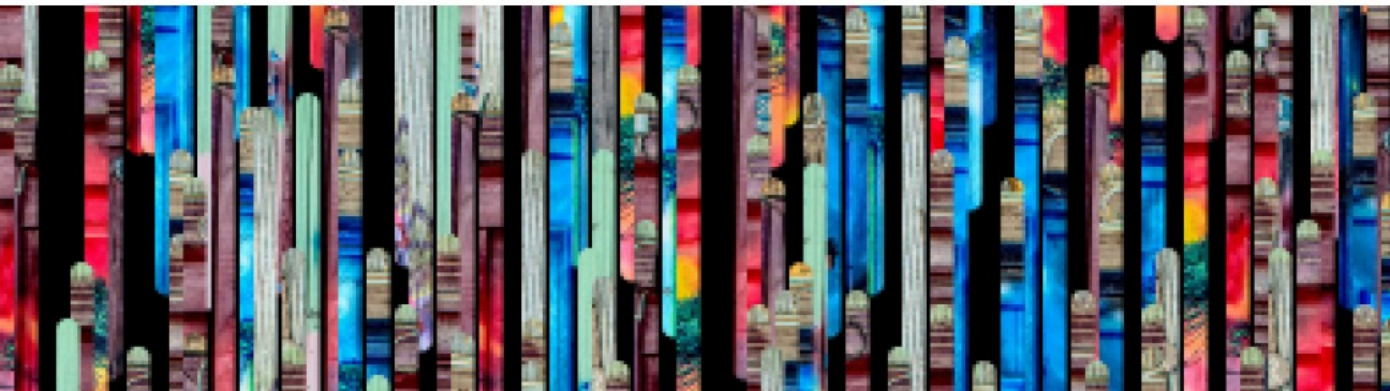
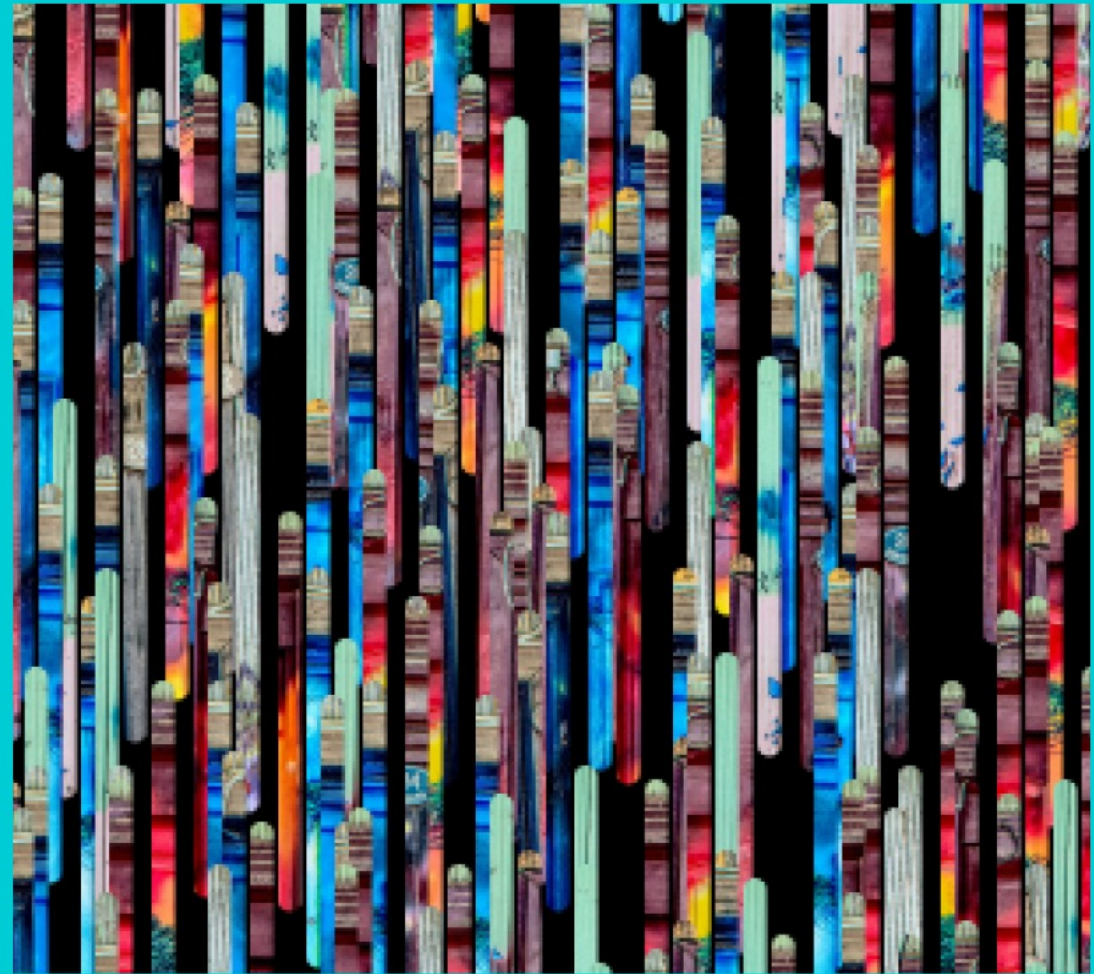




SQL FOR DATA ANALYSTS · LECTURE 1

SQL Basics

Denis Boldin





Contents

- 0. Preliminaries
- 1.1 Filtering and sorting
- 1.2 Aggregate functions
- 1.3 Window functions
- 1.4 Subqueries and CTE
- 1.5 Joins and unions
- 1.6 ACID and the execution plan
- Summary



Contents

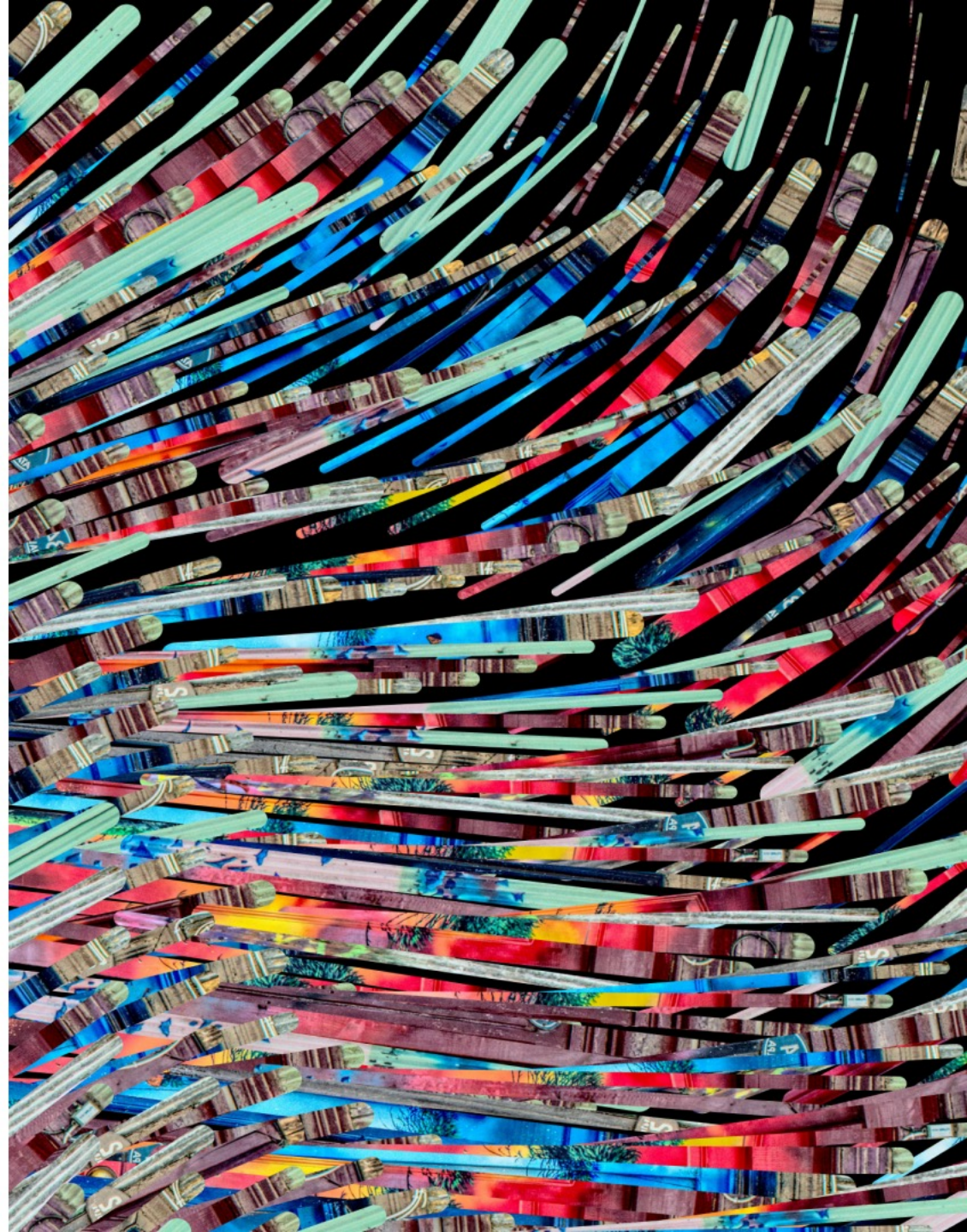
Every example in this lecture runs against the course database — a sample of the Spotify catalogue, 1922 to 2021. Open the [live playground](#) and run them as you read. It is a real PostgreSQL engine in your browser: nothing to install, and a page refresh gives you a clean database back.





0. Preliminaries

- The data
- Data types
- Operators and functions
- The CASE operator
- Type conversion
- Aliases
- Code style





0. PRELIMINARIES

The data

Four tables. Two hold the entities, two hold the relationships between them.

Table	Rows	What one row is
tracks	60,000	one song, with the audio features Spotify computed for it
artists	27,385	one artist, with follower count and genre tags
track_artists	74,624	"this artist is credited on this track"
artist_related	210,844	"listeners of this artist also like that one"



The data

```
artists —< track_artists >— tracks
      |
      |< artist_related >— artists      (an artist linked to other artists)
```



The data

The columns you will use most:

Table	Column	Type	Notes
tracks	name	text	nullable — five tracks in the sample have no name
tracks	popularity	smallint	0–100, Spotify's own score
tracks	duration_ms	integer	milliseconds
tracks	explicit	boolean	
tracks	release_date	date	see the note below
tracks	release_date_precision	text	'day', 'month' or 'year'
tracks	danceability, energy, valence, acousticness, instrumentalness, liveness, speechiness	real	0.0–1.0
tracks	tempo	real	beats per minute
tracks	loudness	real	decibels, normally negative
tracks	key, mode, time_signature	smallint	key 0=C ... 11=B; mode 1=major, 0=minor
artists	followers	bigint	



0. PRELIMINARIES

The data

Table	Column	Type	Notes
artists	popularity	smallint	0–100
artists	genres	text[]	an array , often empty
track_artists	position	smallint	0 = the first credited artist

💡 **A trap worth knowing about now.** The source only knows some release dates to the month or the year. Those are stored padded to the 1st — 1965 becomes 1965-01-01. `release_date_precision` tells you how much of the date to trust. Counting "tracks released on 1 January" would give a wildly wrong answer.



Data types

Most databases have four basic groups of types:

Group	PostgreSQL types
Numeric	SMALLINT, INTEGER, BIGINT, NUMERIC/DECIMAL, REAL, DOUBLE PRECISION
Character	CHAR, VARCHAR(n), TEXT
Date and time	DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL
Flag	BOOLEAN

Plus additional types: arrays (our `genres` is one), `JSON/JSONB`, `UUID`, binary types, geodata.



Data types



Types matter because they:

- **ensure data integrity** — a numeric column will not accept string values;
- **optimize storage** — an integer takes less space than the same value as text;
- **optimize performance** — comparison and sorting are faster on numeric types than on character ones;
- **simplify working with data** — the schema becomes readable and self-documenting.

Types differ between DBMSs, so always check the documentation of the database you are working with.



Operators and functions

A **function** is a subroutine that takes arguments, performs an operation and returns a result. Functions can be nested — the result of one becomes the argument of the next.

Numeric operators and functions:

Operation	Operator	Examples
Addition	+	$3 + 2 \rightarrow 5$ · $3 + 2.0 \rightarrow 5.0$
Subtraction	-	$3 - 2 \rightarrow 1$
Multiplication	*	$3 * 2 \rightarrow 6$
Division	/	$3 / 2 \rightarrow 1$ · $3 / 2.0 \rightarrow 1.50$
Remainder	%	$3 \% 2 \rightarrow 1$
Exponentiation	^	$3 ^ 2 \rightarrow 9$
Absolute value	@	@ -3 → 3



Operators and functions

Purpose	Function	Examples
Remainder of division	<code>mod(numerator, denominator)</code>	<code>mod(5, 2) → 1</code>
Rounding	<code>round(number)</code>	<code>round(1.54) → 2</code>
Nearest larger integer	<code>ceil(number)</code>	<code>ceil(1.54) → 2</code>
Nearest smaller integer	<code>floor(number)</code>	<code>floor(1.54) → 1</code>
Exponentiation	<code>power(number, power)</code>	<code>power(3, 3) → 27</code>
Square root	<code>sqrt(number)</code>	<code>sqrt(1.5) → 1.22</code>



Operators and functions

If you use only integers, the result is an integer. If one argument is fractional, the result is fractional too.

That rule bites immediately on this data. `duration_ms / 60000` gives whole minutes and throws away the rest; `duration_ms / 60000.0` keeps the fraction:

```
SELECT name,  
       duration_ms,  
       duration_ms / 60000      AS minutes_int,  
       duration_ms / 60000.0    AS minutes_exact,  
       round(duration_ms / 60000.0, 2) AS minutes_rounded,  
       (duration_ms || 'milliseconds') :: interval AS interval,  
       to_char((duration_ms || 'milliseconds') :: interval, 'MI:SS') AS minutes_seconds_char  
FROM tracks  
where name = 'Everlong';
```

RESULT

name	duration_ms	minutes_int	minutes_exact	minutes_rounded	interval	minutes_seconds_char
Everlong	250547	4	4.175783333333333	4.18	00:04:10.547	04:10



Operators and functions

String functions:

Purpose	Function	Examples
Concatenation	<code>concat(a, b, c)</code>	<code>concat('Valya had ', 2, ' oranges') → 'Valya had 2 oranges'</code>
String length	<code>length(string)</code>	<code>length('123') → 3</code>
Trimming spaces	<code>trim(string)</code>	<code>trim(' text ') → 'text'</code>
Substring search	<code>position(substring in string)</code>	<code>position('4' in '123') → 0</code>
Upper / lower case	<code>upper(text), lower(text)</code>	<code>upper('text') → 'TEXT'</code>



Operators and functions

Date and time functions:

Purpose	Function	Examples
Current date and time	<code>now()</code>	<code>now()</code> → 2026-05-04 12:44:00
Addition	<code>date + number</code>	<code>date '2001-09-28' + 7</code> → <code>date '2001-10-05'</code>
Subtraction	<code>date - number</code>	<code>date '2001-09-28' - 7</code> → <code>date '2001-09-21'</code>
Difference	<code>date - date</code>	<code>date '2001-10-01' - date '2001-09-28'</code> → 3
Part of a date	<code>extract(part from date)</code>	<code>extract(year from date '2001-10-01')</code> → 2001



Operators and functions

`extract` is the workhorse of this lecture — it is how we get a year or a decade out of `release_date`:

```
SELECT name,  
       release_date,  
       EXTRACT(YEAR FROM release_date)::int      AS year,  
       (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade  
FROM tracks  
ORDER BY release_date  
LIMIT 5;
```

RESULT

name	release_date	year	decade
Maldita sea la primera vez	1900-01-01	1900	1900
Tu Verras Montmartre	1922-01-01	1922	1920
Lady of the Evening	1922-01-01	1922	1920
Nuits De Chine	1922-01-01	1922	1920
Marta	1922-01-01	1922	1920



Operators and functions

Boolean and comparison operators:

Purpose	Operator	Examples
Logical NOT / AND / OR	NOT, AND, OR	true AND false → false
Comparison	=, >, >=, <, <=, <>	7 > 5 → true
Range	BETWEEN	5 BETWEEN 3 AND 7 → true

Operator precedence: comparison operators are evaluated first, then NOT, then AND, and OR last.



The CASE operator

CASE builds conditional expressions, much like `if-else` in a programming language:

```
CASE
  WHEN condition_1 THEN result_1
  WHEN condition_2 THEN result_2
  ...
  ELSE result_N
END
```



0. PRELIMINARIES

The CASE operator

Bucketing tracks by how popular they are:

```
SELECT name,  
       popularity,  
       CASE WHEN popularity >= 80 THEN 'hit'  
            WHEN popularity >= 50 THEN 'known'  
            WHEN popularity >= 20 THEN 'deep cut'  
            ELSE 'obscure' END AS tier,  
       explicit,  
       explicit::int AS explicit_int  
FROM tracks  
ORDER BY popularity DESC, name  
LIMIT 6;
```

RESULT

name	popularity	tier	explicit	explicit_int
Peaches (feat. Daniel Caesar & Giveon)	100	hit	true	1
Blinking Lights	96	hit	false	0
WITHOUT YOU	94	hit	true	1
LA NOCHE DE ANOCHE	93	hit	false	0
DÁKITI	92	hit	true	1
BICHOTA	91	hit	true	1

Note `explicit::int` — a boolean cast to an integer gives 1 or 0, which is handy for summing later.



Type conversion

Conversion can be **implicit** (the DBMS decides) or **explicit** (you decide), using `::` or `CAST`:

```
SELECT CAST('123' AS INTEGER) AS result_1,  
       '123' :: INTEGER      AS result_2;
```

RESULT

result_1	result_2
123	123



Type conversion

💡 "Explicit is better than implicit." Not all DBMSs share the same implicit-conversion rules, and implicit conversion can be slower.

Casting matters on this data because `avg()` over a `real` column returns a `double precision`, and `round(x, 2)` does not accept one — you have to say `::numeric`:

```
round(avg(energy), 2)           -- ERROR: function round(double precision, integer) does not exist
round(avg(energy)::numeric, 2) -- correct
```



0. PRELIMINARIES

Aliases

AS gives a name to whatever an expression or function returns. Technically optional, but without aliases the readability of large queries suffers.





Code style

- Keywords in capital letters (`SELECT`, `FROM`, `WHERE`, `ORDER BY`) — a convention, not a requirement, but a widely accepted one.
- Each clause starts on a new line.
- Align key elements — the "corridor" rule.
- Comment complex queries: `--` for a single line, `/* */` for several.

```
SELECT name, popularity, release_date
FROM tracks
WHERE popularity > 80
ORDER BY release_date
LIMIT 5 OFFSET 10;
```

RESULT

name	popularity	release_date
Hey There Delilah	82	2005-01-01
Fix You	83	2005-06-07
SexyBack (feat. Timbaland)	81	2006-09-12
I'm Yours	83	2008-05-12
Love The Way You Lie	83	2010-06-18



1.1 Filtering and sorting

- Filtering (WHERE)
- Filtering text
- Filtering by a computed field
- Sorting (ORDER BY)
- Limiting the result set (LIMIT, OFFSET)
- Missing data (NULL)
- Check yourself



Filtering (WHERE)

Filtering selects rows by a condition, letting us concentrate only on the data we are interested in.

```
SELECT *           -- choose the fields
FROM table        -- choose the table
WHERE <condition> -- the condition
```



Filtering (WHERE)

The filter expression must return a boolean. Available tools:

- equality and inequality (`=`, `<>`);
- comparisons (`>`, `>=`, `<`, `<=`);
- `IN` — checks whether a value belongs to a list;
- `NOT` — inverts a condition (`NOT IN`, `NOT BETWEEN`, `NOT LIKE`, `IS NOT NULL`);
- `BETWEEN` — checks whether a value falls in a range.





Filtering (WHERE)

Several conditions are combined with **AND** and **OR**; **AND** binds tighter than **OR**.

```
SELECT name, popularity, release_date, explicit
FROM tracks
WHERE explicit
      AND popularity > 75
LIMIT 5;
```

RESULT

name	popularity	release_date	explicit
Creep	83	1993-02-22	true
Still D.R.E.	81	1999-11-16	true
All The Things She Said	76	2002-01-01	true
SexyBack (feat. Timbaland)	81	2006-09-12	true
Flashing Lights	78	2007-09-11	true



Filtering (WHERE)

A `boolean` column needs no comparison — `WHERE explicit` is already a condition. `WHERE explicit = true` works but is noise.

Precedence. These two queries return the same rows — everything from 2020 onwards, plus pre-1950 tracks that are unusually popular — but the second one says so:

```
SELECT count(*)  
FROM tracks  
WHERE release_date >= DATE '2020-01-01'  
OR release_date < DATE '1950-01-01' AND popularity > 50;
```

RESULT

count
2071



Filtering (WHERE)

```
SELECT count(*)  
FROM tracks  
WHERE release_date >= DATE '2020-01-01'  
OR (release_date < DATE '1950-01-01' AND popularity > 50);
```

RESULT

count
2071



1.1 FILTERING AND SORTING

Filtering (WHERE)

Moving the parentheses changes the meaning — now the popularity filter applies to both periods, and the count drops:

```
SELECT count(*)  
FROM tracks  
WHERE (release_date >= DATE '2020-01-01' OR release_date < DATE '1950-01-01')  
AND popularity > 50;
```

RESULT

count
953



To avoid mistakes, set the precedence of logical operators explicitly with parentheses.



1.1 FILTERING AND SORTING

Filtering text

Equality / inequality:

```
SELECT name, popularity FROM tracks WHERE name = 'Imagine' LIMIT 3;
```

RESULT

name	popularity
Imagine	51



1.1 FILTERING AND SORTING

Filtering text

LIKE — search by pattern, where % replaces any number of characters (including zero) and _ replaces exactly one:

```
SELECT name FROM tracks WHERE name LIKE 'The %' LIMIT 3;
```

RESULT

name
The Bridwell Blues
The King of the Zulus
The Sheik of Araby



1.1 FILTERING AND SORTING

Filtering text

```
SELECT name FROM tracks WHERE name LIKE '%Remaster%' LIMIT 3;
```

RESULT

name
El Prisionero - Remasterizado
Practicante - Remasterizado
Sacate la Caretita - Remasterizado



Filtering text

ILIKE — the case-insensitive version (PostgreSQL). Track titles are typed by humans, so case is never reliable:

```
SELECT name, popularity
FROM tracks
WHERE name ILIKE '%love%'
ORDER BY popularity DESC
LIMIT 5;
```

RESULT

name	popularity
What You Know Bout Love	91
i love you	83
Another Love	83
Love The Way You Lie	83
Lasting Lover	83



Filtering text

IN — matches one of a list of values:

```
SELECT name, key, mode, time_signature
FROM tracks
WHERE time_signature IN (3, 5, 7)
AND popularity > 70
LIMIT 5;
```

RESULT

name	key	mode	time_signature
What A Wonderful World	5	1	3
Annie's Song	2	1	3
Somebody To Love - Remastered 2011	8	1	3
Crazy	6	0	3
Ironic - 2015 Remaster	11	1	5



1.1 FILTERING AND SORTING

Filtering text

Text functions in the filter — `LOWER()`, `UPPER()`, `TRIM()`, `SUBSTRING()`:

```
SELECT name
FROM tracks
WHERE LOWER(name) = 'yesterday'
LIMIT 3;
```

RESULT

name
Yesterday
Yesterday
Yesterday



Filtering text

💡 Wrapping a column in a function stops PostgreSQL using an ordinary index on it. `LOWER(name) = 'yesterday'` cannot use an index on `name`. That does not matter on 60,000 rows; it matters a great deal on 60,000,000. Lecture 3 returns to this.

Conditions combine freely:

```
SELECT name, popularity, tempo
FROM tracks
WHERE (name ILIKE '%love%' OR name ILIKE '%heart%')
AND popularity > 60
AND tempo BETWEEN 100 AND 130
LIMIT 5;
```

RESULT

name	popularity	tempo
Dedicated To The One I Love	65	103.264
Satellite of Love	61	121.861
Can't Get Enough Of Your Love, Babe	69	111.833
This Will Be (An Everlasting Love)	74	126.739
Say You Love Me	64	128.068



Filtering by a computed field

`WHERE` cannot see the aliases defined in `SELECT`, so the expression has to be repeated:

```
-- correct
SELECT name, duration_ms, duration_ms / 60000.0 AS minutes
FROM tracks
WHERE duration_ms / 60000.0 > 30
ORDER BY duration_ms DESC
LIMIT 5;
```

RESULT

name	duration_ms	minutes
Happy New Year Mix 2009	4696690	78.27816666666667
Sukhmani Sahib	4263114	71.05190000000000
MINIMAL NEW YEAR 2017 - Continuous DJ Mix	3807002	63.45003333333333
SkyTop 2018 Year Mix - Continuous DJ Mix	3693390	61.55650000000000
Surah Al-Baqara, Pt. 1	3616000	60.26666666666667



Filtering by a computed field

```
-- error: the alias is not visible in WHERE  
SELECT name, duration_ms, duration_ms / 60000.0 AS minutes  
  FROM tracks  
 WHERE minutes > 30;
```

💡 Do not use aliases for filtering by computed fields.



Sorting (ORDER BY)

ORDER BY orders rows by one or more columns; **ASC** (the default) ascending, **DESC** descending.

```
SELECT *  
  FROM table  
 WHERE <condition>  
ORDER BY <field_1> DESC, <field_2>
```



Sorting (ORDER BY)

The sort field can be given by name, by alias, or by its ordinal number among the output fields (starting from 1). Numbers sort numerically, character types lexicographically.

Unlike `WHERE`, `ORDER BY` can see a `SELECT` alias, because sorting happens after the columns are computed (see the order of execution):

```
SELECT name,  
       round(duration_ms / 60000.0, 2) AS minutes  
FROM tracks  
WHERE popularity > 70  
ORDER BY minutes DESC  
LIMIT 5;
```

RESULT

name	minutes
Poesia Acústica 10: Recomeçar	11.38
Poesia Acústica #6: Era uma Vez	9.63
Mirrors	8.07
Comfortably Numb	6.37
Sympathy For The Devil - 50th Anniversary Edition	6.30



Limiting the result set (LIMIT, OFFSET)

LIMIT restricts the number of rows returned; combined with **OFFSET** it gives pagination.

```
SELECT *  
  FROM table  
  WHERE <condition>  
ORDER BY <field_1>, <field_2>  
  LIMIT n1                -- return n1 rows  
  OFFSET n2               -- skip the first n2 rows
```



Limiting the result set (LIMIT, OFFSET)

`LIMIT 5 OFFSET 5` selects 5 rows starting from the 6th — the second page:

```
SELECT name, popularity
FROM tracks
ORDER BY popularity DESC, name
LIMIT 5 OFFSET 5;
```

RESULT

name	popularity
BICHOTA	91
What You Know Bout Love	91
Anyone	90
Goosebumps	89
La Nota	89



Limiting the result set (LIMIT, OFFSET)

💡 LIMIT without ORDER BY gives you *some* rows, not the *first* rows — the database is free to return them in any order, and that order can change between runs. Always pair LIMIT with ORDER BY when the choice matters.



Missing data (NULL)

NULL denotes the absence of data. It is not an empty string and not zero, and it needs a special approach.

This database has real **NULL**s to practise on. Five tracks arrived with no title at all:

NULL does not take part in ordinary comparisons — use **IS NULL** / **IS NOT NULL**:

```
SELECT id, name, popularity, release_date
FROM tracks
WHERE name IS NULL;
```

RESULT

id	name	popularity	release_date
5Wjlz5WSaTuiDo2VoncxnO	NULL	0	1922-04-01
3PzaDPPZalpAybEBqWiOpC	NULL	0	1934-04-01
3XrydbtC0OcMLbliPqR8KV	NULL	0	1934-04-01
30XNxfbijiZ1E8SoLgAfC3	NULL	0	1947-04-01
0dXn9K8NH2qn1jvG2dMQ8Q	NULL	0	1950-01-01



Missing data (NULL)

`WHERE name = NULL` would return nothing at all — not an error, just zero rows, which is exactly the kind of bug that survives to production.

Arithmetic and concatenation with `NULL` yield `NULL`:

```
SELECT id,  
       name,  
       name || ' (' || popularity || ')' AS label  
FROM tracks  
WHERE name IS NULL  
LIMIT 3;
```

RESULT

id	name	label
5Wjlz5WSaTuiDo2VoncxnO	NULL	NULL
3PzaDPPZalpAybEBqWiOpC	NULL	NULL
3XrydbtC0OcMLbliPqR8KV	NULL	NULL



Missing data (NULL)

COALESCE returns the first non-NULL argument:

```
SELECT id,  
       COALESCE(name, '(untitled)') AS name,  
       popularity  
FROM tracks  
WHERE name IS NULL;
```

RESULT

id	name	popularity
5Wjlz5WSaTuiDo2VoncxnO	(untitled)	0
3PzaDPPZalpAybEBqWiOpC	(untitled)	0
3XrydbtC0OcMLbliPqR8KV	(untitled)	0
30XNxfbijiZ1E8SoLgAfC3	(untitled)	0
0dXn9K8NH2qn1jvG2dMQ8Q	(untitled)	0



Missing data (NULL)

Sorting — **NULL** may come first or last depending on the DBMS; state it with **NULLS FIRST** / **NULLS LAST**:

```
SELECT id, name, popularity
FROM tracks
ORDER BY name NULLS FIRST
LIMIT 8;
```

RESULT

id	name	popularity
3XrydbtC0OcMLbliPqR8KV	NULL	0
3PzaDPPZalpAybEBqWiOpC	NULL	0
30XNxfbijiZ1E8SoLgAfC3	NULL	0
0dXn9K8NH2qn1jvG2dMQ8Q	NULL	0
5Wjlz5WSaTuiDo2VoncxnO	NULL	0
0gNNTToCW3qjabgTyBSjt3H	!Que Vida! - Mono Version	23
26ENGK2mGjJPvE858BTmPE	"43"	30
1zMPEr35vNUdXwgtAVaPq0	"Autoportrait, je ne comprends pas..."	0



Missing data (NULL)

An empty array is not **NULL**. `artists.genres` is a `text[]`, and thousands of artists have no tags. The array is empty (`{}`), but `array_length` of an empty array returns **NULL** — a distinction that trips people up:

```
SELECT count(*)                AS artists_total,  
       count(array_length(genres, 1)) AS with_genres,  
       count(*) FILTER (WHERE genres = '{}') AS empty_array  
FROM artists;
```

RESULT

artists_total	with_genres	empty_array
27385	21659	5726

`count(*)` counts rows; `count(<expression>)` counts rows where the expression is not **NULL**. The gap between the two columns *is* the number of untagged artists.



Check yourself

What will `NULL = NULL` return? — `NULL`: a comparison with `NULL` is always undefined.

What will `NULL OR TRUE` return? — `TRUE`: `OR` is true if at least one operand is true.

What will `NULL AND TRUE` return? — `NULL`: the result depends on the unknown value.

How is `NULL` handled during sorting? — differently in different DBMSs; change it with `NULLS FIRST` / `NULLS LAST`.

Why does `WHERE name = NULL` return no rows instead of an error? — it is valid SQL; the comparison simply evaluates to `NULL` for every row, and `NULL` is not `TRUE`.

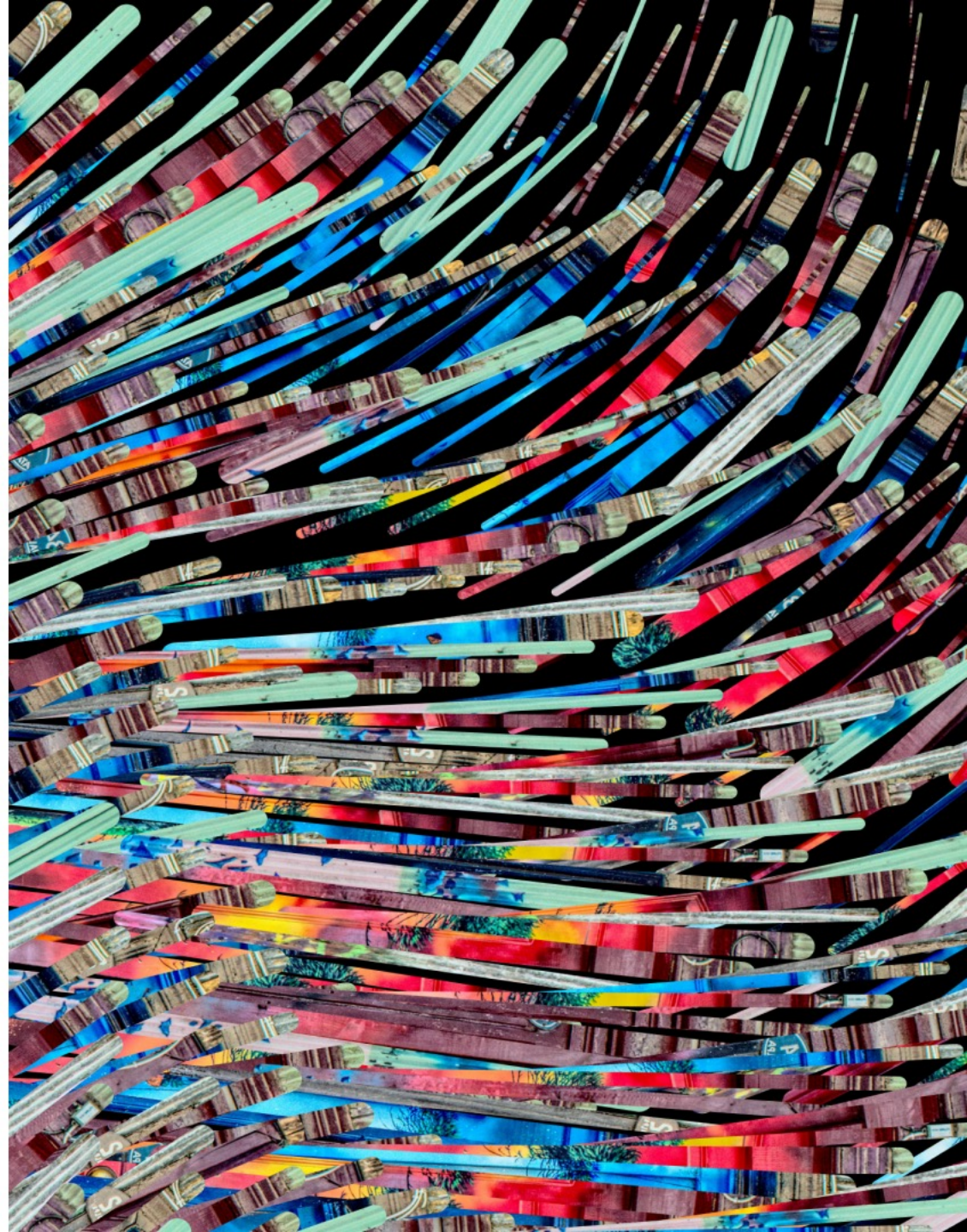




1.2 Aggregate functions

Aggregate functions perform a calculation over a set of values and return a single value — the aggregate. They are used to analyse and summarise data: the sum of today's purchases, the number of people in a department, the average length of a song.

- Specifics of COUNT
- Counting several things in one pass
- Grouping data (GROUP BY)
- Filtering by aggregates (HAVING)
- Check yourself





1.2 Aggregate functions

Purpose	Function	Example
Count	COUNT(<field>)	SELECT COUNT(*) FROM tracks
Sum	SUM(<field>)	SELECT SUM(duration_ms) FROM tracks
Average	AVG(<field>)	SELECT AVG(popularity) FROM tracks
Minimum	MIN(<field>)	SELECT MIN(release_date) FROM tracks
Maximum	MAX(<field>)	SELECT MAX(tempo) FROM tracks



1.2 Aggregate functions

```
SELECT count(*)           AS tracks,  
       round(avg(popularity), 2) AS avg_popularity,  
       min(release_date)   AS earliest,  
       max(release_date)   AS latest,  
       round(sum(duration_ms) / 3600000.0) AS total_hours  
FROM tracks;
```

RESULT

tracks	avg_popularity	earliest	latest	total_hours
60000	27.60	1900-01-01	2021-04-16	3827



Specifics of COUNT

- `COUNT(*)` — the number of rows in the table (or in the filtered table, if `WHERE` is used);
- `COUNT(1)` — the same as `COUNT(*)`, but the DBMS does not read the values of every field, using a constant instead;
- `COUNT(<field>)` — the number of non-empty values in the field;
- `COUNT(DISTINCT <field>)` — the number of unique non-empty values in the field.

The five nameless tracks make the difference visible:

```
SELECT count(*)           AS count_star,  
       count(1)           AS count_1,  
       count(name)        AS count_name,  
       count(DISTINCT name) AS count_distinct_name,  
       count(DISTINCT time_signature) AS count_distinct_time_sig  
FROM tracks;
```

RESULT

count_star	count_1	count_name	count_distinct_name	count_distinct_time_sig
60000	60000	59995	56448	5



Specifics of COUNT

`count_star` and `count_name` differ by exactly the five `NULL` titles. `count_distinct_name` is far lower still — the catalogue is full of remasters, live versions and re-releases sharing a title.





Counting several things in one pass

`COUNT` plus `CASE` counts values of different kinds without a `UNION` and without grouping:

```
SELECT count(1) AS row_cnt,  
       SUM(CASE WHEN explicit THEN 1 ELSE 0 END) AS explicit_cnt,  
       SUM(CASE WHEN NOT explicit THEN 1 ELSE 0 END) AS clean_cnt  
FROM tracks  
WHERE release_date >= DATE '2010-01-01';
```

RESULT

row_cnt	explicit_cnt	clean_cnt
12832	1810	11022



Counting several things in one pass

PostgreSQL offers a cleaner spelling for the same idea — `FILTER`:

```
SELECT count(*)                               AS row_cnt,  
       count(*) FILTER (WHERE explicit)       AS explicit_cnt,  
       count(*) FILTER (WHERE NOT explicit)   AS clean_cnt,  
       round(100.0 * count(*) FILTER (WHERE explicit) / count(*), 1) AS pct_explicit  
FROM tracks  
WHERE release_date >= DATE '2010-01-01';
```

RESULT

row_cnt	explicit_cnt	clean_cnt	pct_explicit
12832	1810	11022	14.1

`FILTER` says what it means, and it works with any aggregate, not just `COUNT`.



Grouping data (GROUP BY)

GROUP BY lets you specify the grouping field and execute aggregate functions within each group.

```
SELECT <grouping field>, <aggregate>  
FROM table  
GROUP BY <grouping field>;
```



Grouping data (GROUP BY)

The single most useful grouping on this data is by decade:

```
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,  
       count(*) AS tracks,  
       round(avg(popularity), 1) AS avg_popularity,  
       round(avg(duration_ms) / 60000.0, 2) AS avg_minutes,  
       round(avg(energy)::numeric, 3) AS avg_energy  
FROM tracks  
GROUP BY decade  
ORDER BY decade;
```



Grouping data (GROUP BY)

RESULT

decade	tracks	avg_popularity	avg_minutes	avg_energy
1900	1	19.0	3.90	0.791
1920	778	1.1	2.99	0.285
1930	1333	2.0	3.54	0.308
1940	1845	1.8	3.65	0.271
1950	3617	8.3	3.68	0.296
1960	4834	18.2	3.48	0.407
1970	6325	24.2	3.91	0.503
1980	8419	25.9	3.81	0.549
1990	11135	29.3	4.01	0.571
2000	8881	36.6	4.05	0.648
2010	10764	39.3	3.86	0.662
2020	2068	41.1	3.33	0.634



Grouping data (GROUP BY)

Read down the `avg_energy` column: recorded music gets steadily louder and more energetic over a century. That is one `GROUP BY` away, and it is the kind of result that makes a stakeholder sit up.

A grouping key does not have to be a bare column — it can be any expression, as here. You may also refer to it by its output position, `GROUP BY 1`, which is shorter but harder to read when the query grows.



Filtering by aggregates (HAVING)

Sometimes you need to filter data *after* it has been aggregated. That is what **HAVING** is for.

```
SELECT <grouping field>, <aggregate>
FROM table
GROUP BY <grouping field>
HAVING <condition on the aggregate>
```

Which artists have a substantial catalogue in this sample?



Filtering by aggregates (HAVING)

```
SELECT a.name,  
       count(*) AS tracks  
FROM artists AS a  
JOIN track_artists AS ta ON ta.artist_id = a.id  
GROUP BY a.name  
HAVING count(*) > 150  
ORDER BY tracks DESC;
```

RESULT

name	tracks
Die drei ???	392
Lata Mangeskar	282
Francisco Canaro	210
TKKG Retro-Archiv	210
Johann Sebastian Bach	195
Wolfgang Amadeus Mozart	188
Bibi Blocksberg	163
Benjamin Blümchen	163



Filtering by aggregates (HAVING)

Filtering by *non-aggregated* values can be written either in **WHERE** or in **HAVING** — these two queries return the same result:

```
SELECT explicit, count(*)  
  FROM tracks  
 WHERE release_date >= DATE '2015-01-01'  
 GROUP BY explicit;
```

RESULT

explicit	count
false	6164
true	1390



Filtering by aggregates (HAVING)

```
SELECT explicit, count(*)  
FROM tracks  
GROUP BY explicit, release_date  
HAVING release_date >= DATE '2015-01-01'  
LIMIT 3;
```

RESULT

explicit	count
false	3
false	4
true	2



Filtering by aggregates (HAVING)

But you should filter non-aggregated data in `WHERE`, that is, in advance: unnecessary rows are removed before grouping, and computing resources are not spent counting values you are going to throw away anyway. The second query above also had to add `release_date` to the grouping key to make it legal at all — a good sign the filter was in the wrong place.



Check yourself

How do you count the number of tracks released in the 1990s? — `COUNT(*)` with a `WHERE` on `release_date`.

How do you count the total number of tracks, the explicit ones and the clean ones in one query? — `COUNT(*)` plus `COUNT(*) FILTER (WHERE ...)` per case, or `SUM(CASE ...)`.

How can you find duplicates using grouping? — group by the fields that should be unique, count the rows per group, and keep the groups with `HAVING COUNT(1) > 1`.

What is the difference between `WHERE` and `HAVING`? — `WHERE` filters the source rows before grouping, `HAVING` filters aggregates after it. Order of execution: `WHERE` → `GROUP BY` → `HAVING`.

Why does `COUNT(name)` differ from `COUNT(*)` here? — `COUNT(<field>)` skips `NULL`s, and five tracks have no name.



1.3 Window functions

An **analytic (window) function** performs calculations over a set of records forming a window (partition), and returns a single value per row.

- Syntax
- The base query for the examples
- The three groups of analytic functions
- Aggregate window functions
- Offset functions
- Ranking functions
- Frames
- WINDOW — a named window
- Check yourself



1.3 Window functions

💡 The main advantage of window functions: they return exactly as many records as they received. With `GROUP BY`, a group collapses into one row. A window function performs the same aggregation by group but preserves the structure of the source table — the aggregation result is simply added to every record in a separate column.

Data is divided into **partitions** by a field or fields; inside each partition it is sorted and then analysed.





Syntax

```
<function>(<field>) OVER (PARTITION BY <partition> ORDER BY <sorting> <frame>)
```

- **function** — the window function over the chosen field;
- **partition** — the field or set of fields defining the group;
- **sorting** — the field or fields by which rows are ordered inside the partition;
- **frame** — the set of rows inside the window we operate on (optional).

First the data is split into partitions, then sorted, and only then is the function applied to each record inside the partition.



The base query for the examples

Everything below builds on the same small result: how many tracks two artists released per decade. It is deliberately tiny so you can check the arithmetic by eye.

```
WITH artist_decade AS (  
    SELECT a.name  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name IN ('David Bowie', 'Bob Dylan')  
    GROUP BY 1, 2  
)  
SELECT * FROM artist_decade ORDER BY artist, decade;
```

RESULT

artist	decade	track_cnt
Bob Dylan	1960	22
Bob Dylan	1970	16
Bob Dylan	1980	4
David Bowie	1960	3
David Bowie	1970	14
David Bowie	1980	6
David Bowie	1990	1
David Bowie	2010	8

The WITH block is a CTE, covered in 1.4. Read it for now as "give this query a name". Every example below repeats it so you can paste any of them straight into the playground.



The three groups of analytic functions

AGGREGATION	OFFSET	RANKING
AVG()	FIRST_VALUE()	ROW_NUMBER()
COUNT()	LAST_VALUE()	RANK()
MAX()	LAG()	DENSE_RANK()
MIN()	LEAD()	NTILE()
SUM()	NTH_VALUE()	CUME_DIST()
		PERCENT_RANK()



Aggregate window functions

AVG, COUNT, MAX, MIN, SUM — cumulative values inside a window:

```
WITH artist_decade AS (  
    SELECT a.name AS artist,  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name IN ('David Bowie', 'Bob Dylan')  
    GROUP BY 1, 2  
)  
SELECT artist,  
       decade,  
       track_cnt,  
       SUM(track_cnt) OVER w ::int           AS running_sum,  
       round(AVG(track_cnt) OVER w, 2)       AS running_avg,  
       MIN(track_cnt) OVER w ::int           AS running_min,  
       MAX(track_cnt) OVER w ::int           AS running_max  
FROM artist_decade  
WINDOW w AS (PARTITION BY artist ORDER BY decade)  
ORDER BY artist, decade;
```



Aggregate window functions

RESULT

artist	decade	track_cnt	running_sum	running_avg	running_min	running_max
Bob Dylan	1960	22	22	22.00	22	22
Bob Dylan	1970	16	38	19.00	16	22
Bob Dylan	1980	4	42	14.00	4	22
David Bowie	1960	3	3	3.00	3	3
David Bowie	1970	14	17	8.50	3	14
David Bowie	1980	6	23	7.67	3	14
David Bowie	1990	1	24	6.00	1	14
David Bowie	2010	8	32	6.40	1	14



Aggregate window functions

💡 Why is the sum computed over the current row and all previous ones rather than over all of the artist's decades? Because when a window function is used with an aggregate, a **window frame** is determined for each row. If you specify `ORDER BY` inside `OVER`, the frame by default runs from the beginning of the partition to the current row — hence the running total. Without `ORDER BY`, the frame is the whole partition. Without `PARTITION BY`, the frame is the whole table.



Aggregate window functions

Compare — the same `SUM` with no `ORDER BY` gives each artist's grand total on every row:

```
WITH artist_decade AS (  
    SELECT a.name AS artist,  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name IN ('David Bowie', 'Bob Dylan')  
    GROUP BY 1, 2  
)  
SELECT artist,  
       decade,  
       track_cnt,  
       SUM(track_cnt) OVER (PARTITION BY artist)::int AS artist_total,  
       round(100.0 * track_cnt / SUM(track_cnt) OVER (PARTITION BY artist), 1) AS pct_of_career  
FROM artist_decade  
ORDER BY artist, decade;
```



Aggregate window functions

RESULT

artist	decade	track_cnt	artist_total	pct_of_career
Bob Dylan	1960	22	42	52.4
Bob Dylan	1970	16	42	38.1
Bob Dylan	1980	4	42	9.5
David Bowie	1960	3	32	9.4
David Bowie	1970	14	32	43.8
David Bowie	1980	6	32	18.8
David Bowie	1990	1	32	3.1
David Bowie	2010	8	32	25.0

"What share of the total does this row represent" is one of the most common questions in analytics, and this is the shape of the answer.



Offset functions

Offset functions reach into neighbouring rows:

- **FIRST_VALUE** — the first row in the frame
- **LAST_VALUE** — the last row in the frame
- **LAG** — the value in the previous row of the partition
- **LEAD** — the value in the next row of the partition
- **NTH_VALUE** — the value in row number N (the second argument)

```
WITH artist_decade AS (  
    SELECT a.name AS artist,  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name IN ('David Bowie', 'Bob Dylan')  
    GROUP BY 1, 2  
)  
SELECT artist,  
       decade,  
       track_cnt,  
       FIRST_VALUE(track_cnt) OVER w AS first_decade,  
       LAG(track_cnt) OVER w AS prev_decade,  
       LEAD(track_cnt) OVER w AS next_decade,  
       track_cnt - LAG(track_cnt) OVER w AS change,  
       NTH_VALUE(track_cnt, 2) OVER w AS second_decade  
FROM artist_decade  
WINDOW w AS (PARTITION BY artist ORDER BY decade)  
ORDER BY artist, decade;
```



Offset functions

RESULT

artist	decade	track_cnt	first_decade	prev_decade	next_decade	change	second_decade
Bob Dylan	1960	22	22	NULL	16	NULL	NULL
Bob Dylan	1970	16	22	22	4	-6	16
Bob Dylan	1980	4	22	16	NULL	-12	16
David Bowie	1960	3	3	NULL	14	NULL	NULL
David Bowie	1970	14	3	3	6	11	14
David Bowie	1980	6	3	14	1	-8	14
David Bowie	1990	1	3	6	8	-5	14
David Bowie	2010	8	3	1	NULL	7	14

LAG is how you compute any change-over-time: the value minus the previous value. Note the **NULL** in the first row of each partition — there is no previous row, so the difference is unknown, not zero.



Ranking functions

- **ROW_NUMBER** — the row number inside the partition
- **RANK** — the rank within the partition, skipping the next rank when values repeat
- **DENSE_RANK** — the rank without gaps when values repeat
- **NTILE** — splits the window into the given number of groups and returns the group number
- **CUME_DIST** — the cumulative distribution, RANK/COUNT
- **PERCENT_RANK** — the relative rank, $(\text{RANK}-1)/(\text{COUNT}-1)$

Ties are what separate these functions, and popularity scores are full of ties:

```
WITH bowie AS (  
    SELECT t.name, t.popularity  
    FROM tracks AS t  
    JOIN track_artists AS ta ON ta.track_id = t.id  
    JOIN artists AS a ON a.id = ta.artist_id  
    WHERE a.name = 'David Bowie'  
    ORDER BY t.popularity DESC, t.name  
    LIMIT 8  
)  
SELECT name,  
    popularity,  
    ROW_NUMBER() OVER w AS rn,  
    RANK() OVER w AS rank,  
    DENSE_RANK() OVER w AS dense_rank,  
    NTILE(2) OVER w AS ntile_2,  
    round((CUME_DIST() OVER w)::numeric, 2) AS cume_dist,  
    round((PERCENT_RANK() OVER w)::numeric, 2) AS percent_rank  
FROM bowie  
WINDOW w AS (ORDER BY popularity DESC);
```



Ranking functions

RESULT

name	popularity	rn	rank	dense_rank	ntile_2	cume_dist	percent_rank
Moonage Daydream - 2012 Remaster	69	1	1	1	1	0.13	0.00
Sound and Vision - 2017 Remaster	63	2	2	2	1	0.25	0.14
Under Pressure	59	3	3	3	1	0.38	0.29
As The World Falls Down	55	4	4	4	1	0.50	0.43
Tonight (With David Bowie) - Live	48	5	5	5	2	0.63	0.57
Cracked Actor - 2013 Remaster	43	6	6	6	2	0.88	0.71
Word on a Wing - 2016 Remaster	43	7	6	6	2	0.88	0.71
Beauty and the Beast - 2017 Remaster	42	8	8	7	2	1.00	1.00



Ranking functions

- `ROW_NUMBER` numbers every row, breaking ties arbitrarily; `NTILE(2)` splits the window into two roughly equal halves.
- Where two tracks share a popularity score they get the same `RANK` and the same `DENSE_RANK`. The row after a tie is where they diverge: `RANK` skips a number, `DENSE_RANK` does not.
- `CUME_DIST` shows the share of rows less than **or equal to** the current row; `PERCENT_RANK` the share of rows strictly less than it. Hence the last value of both is always 1, while the first value is 0 for `PERCENT_RANK` but non-zero for `CUME_DIST`.





Ranking functions

The most common use of **ROW_NUMBER**: **top-N per group**. Rank inside each partition, then filter on the rank in an outer query — you cannot filter on a window function in **WHERE**, because windows are computed after **WHERE** runs:

```
WITH ranked AS (  
    SELECT name,  
           popularity,  
           (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,  
           ROW_NUMBER() OVER (  
               PARTITION BY (EXTRACT(YEAR FROM release_date)::int / 10) * 10  
               ORDER BY popularity DESC, name  
           ) AS rnk  
    FROM tracks  
)  
SELECT decade, rnk, name, popularity  
    FROM ranked  
 WHERE rnk <= 2  
 ORDER BY decade, rnk;
```



1.3 WINDOW FUNCTIONS

Ranking functions

RESULT

decade	rnk	name	popularity
1900	1	Maldita sea la primera vez	19
1920	1	St. James Infirmary	30
1920	2	All My Life - Live	29
1930	1	Gloomy Sunday (with Teddy Wilson & His Orchestra) - Take 1	51
1930	2	Sweet Home Chicago	46
1940	1	Nancy (With the Laughing Face) - 78rpm Version	55
1940	2	Saturday Night (Is The Loneliest Night In The Week)	54
1950	1	I've Got You Under My Skin - Remastered 1998	68
1950	2	Lonesome Town	67
1960	1	Fortunate Son	83
1960	2	(I Can't Get No) Satisfaction - Mono Version	78
1970	1	Mr. Blue Sky	82

decade	rnk	name	popularity
1970	2	More Than a Feeling	80
1980	1	Summer Of '69	83
1980	2	Sweet Child O' Mine	82
1990	1	Creep	83
1990	2	Losing My Religion	83
2000	1	Fix You	83
2000	2	I'm Yours	83
2010	1	Dance Monkey	88
2010	2	SAD!	87
2020	1	Peaches (feat. Daniel Caesar & Giveon)	100
2020	2	Blinding Lights	96



Frames

Frames specify the exact range of rows to aggregate over. Two ways of defining them:

1. **ROWS** — works with rows inside the partition.
2. **RANGE** — works with a range of values inside the partition.

ROWS — boundaries by row number:

ROWS BETWEEN <lower boundary> **AND** <upper boundary>

- **UNBOUNDED PRECEDING** — the window starts at the first row of the group
- **UNBOUNDED FOLLOWING** — the window ends at the last row of the group
- **CURRENT ROW** — the window starts or ends at the current row
- **<n> PRECEDING / <n> FOLLOWING** — n rows before / after the current row



Frames

```
WITH artist_decade AS (  
    SELECT a.name AS artist,  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name = 'David Bowie'  
    GROUP BY 1, 2  
)  
SELECT decade,  
       track_cnt,  
       SUM(track_cnt) OVER (ORDER BY decade  
                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)::int AS v1,  
       SUM(track_cnt) OVER (ORDER BY decade  
                           ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)::int AS v2,  
       SUM(track_cnt) OVER (ORDER BY decade  
                           ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)::int AS v3  
FROM artist_decade  
ORDER BY decade;
```

RESULT

decade	track_cnt	v1	v2	v3
1960	3	3	32	17
1970	14	17	29	23
1980	6	23	15	21
1990	1	24	9	15
2010	8	32	8	9



Frames

`v1` is the running total, `v2` the total still to come, `v3` a three-row moving window — the standard smoothing trick for a noisy time series.

RANGE — boundaries by row *values* rather than row numbers:

RANGE BETWEEN <lower boundary> **AND** <upper boundary>



Frames

💡 If no frame is specified, **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** applies: all records from the first row of the window up to the current one.

The difference shows up on ties. Ordering by **track_cnt**, **ROWS** treats two equal values as two separate steps, while **RANGE** folds them into one:

```
WITH artist_decade AS (  
  SELECT a.name AS artist,  
         (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
         count(*) AS track_cnt  
  FROM artists AS a  
  JOIN track_artists AS ta ON ta.artist_id = a.id  
  JOIN tracks AS t ON t.id = ta.track_id  
  WHERE a.name IN ('David Bowie', 'Bob Dylan')  
  GROUP BY 1, 2  
)  
SELECT artist,  
       decade,  
       track_cnt,  
       SUM(track_cnt) OVER (PARTITION BY artist ORDER BY track_cnt  
                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)::int AS by_rows,  
       SUM(track_cnt) OVER (PARTITION BY artist ORDER BY track_cnt  
                           RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)::int AS by_range  
FROM artist_decade  
ORDER BY artist, track_cnt;
```



Frames

RESULT

artist	decade	track_cnt	by_rows	by_range
Bob Dylan	1980	4	4	4
Bob Dylan	1970	16	20	20
Bob Dylan	1960	22	42	42
David Bowie	1990	1	1	1
David Bowie	1960	3	4	4
David Bowie	1980	6	10	10
David Bowie	2010	8	18	18
David Bowie	1970	14	32	32



WINDOW — a named window

A named window can be reused several times in one query. It improves readability, and in some DBMSs speeds the query up. We have been using it above; here it is on its own:

```
WINDOW <alias> AS (PARTITION BY <partition> ORDER BY <sorting> <frame>)
```



WINDOW — a named window

```
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,  
       count(*) AS tracks,  
       SUM(count(*)) OVER w ::int AS running_tracks,  
       round(AVG(count(*)) OVER w, 1) AS running_avg  
FROM tracks  
GROUP BY decade  
WINDOW w AS (ORDER BY (EXTRACT(YEAR FROM release_date)::int / 10) * 10)  
ORDER BY decade;
```



WINDOW — a named window

RESULT

decade	tracks	running_tracks	running_avg
1900	1	1	1.0
1920	778	779	389.5
1930	1333	2112	704.0
1940	1845	3957	989.3
1950	3617	7574	1514.8
1960	4834	12408	2068.0
1970	6325	18733	2676.1
1980	8419	27152	3394.0
1990	11135	38287	4254.1
2000	8881	47168	4716.8
2010	10764	57932	5266.5
2020	2068	60000	5000.0



WINDOW — a named window

Note what just happened: `SUM(count(*)) OVER (...)` is a window function applied to an aggregate. It is legal because windows are evaluated *after* `GROUP BY`, so by the time the window runs, `count(*)` is just another column.



Check yourself

How do you compute a cumulative sum? — with the analytic `SUM`: partition the data, sort it, apply the aggregate.

What happens if you do not specify a frame? — aggregation runs from the first row of the window up to and including the current one.

What options are there for specifying a frame? — by rows (`ROWS`) and by row values (`RANGE`).

What is the difference between `RANK` and `DENSE_RANK`? — `RANK` leaves gaps after ties; `DENSE_RANK` does not.

Why can't you write `WHERE ROW_NUMBER() OVER (...) <= 3`? — `WHERE` is evaluated before window functions. Wrap the query in a CTE or subquery and filter outside.

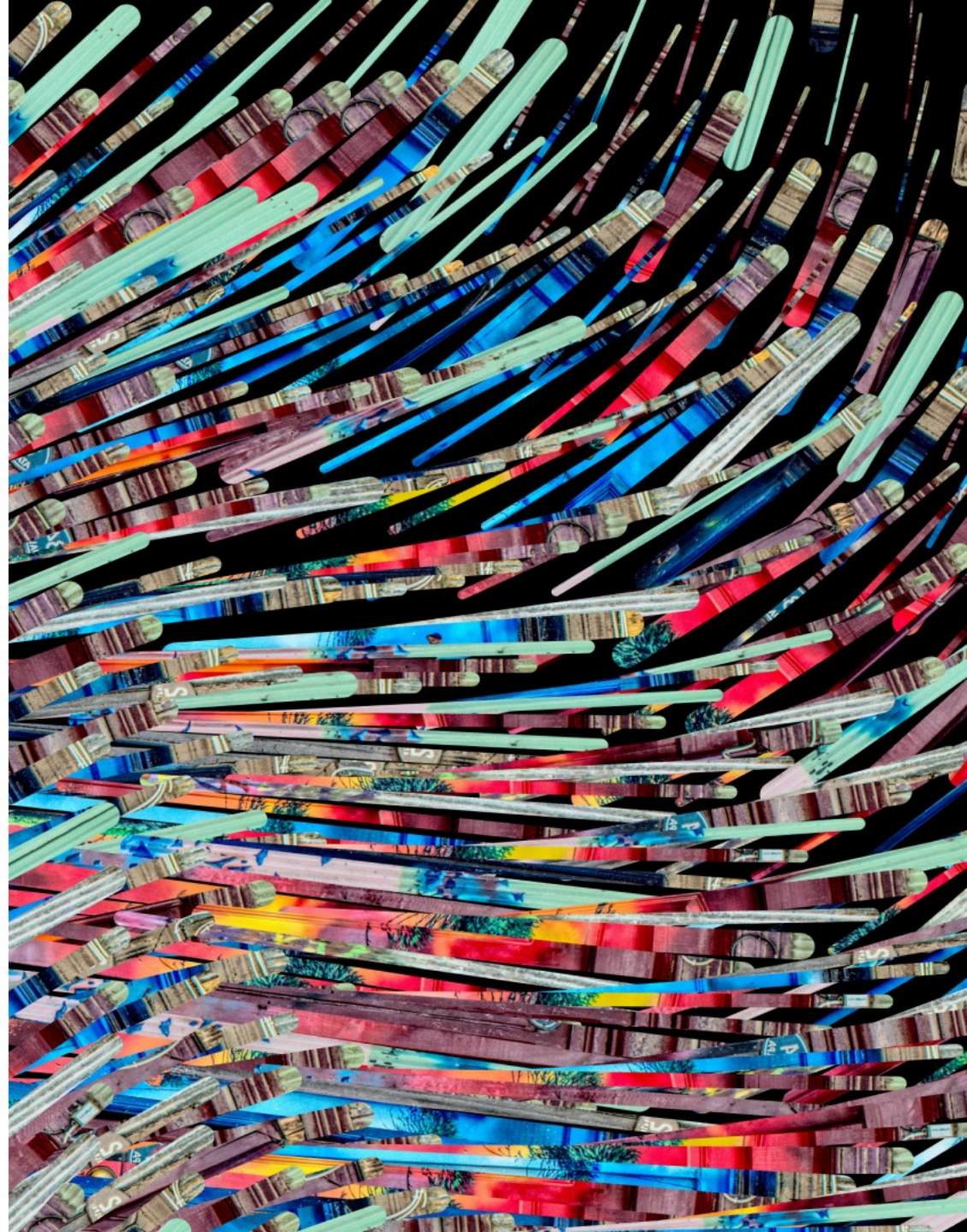




1.4 Subqueries and CTE

A **subquery** is a query inside another query. Subqueries can appear in **SELECT**, **FROM**, **JOIN**, **WHERE**, **HAVING** and **ORDER BY**.

- Types of subqueries
- Subqueries in FROM
- Subqueries in SELECT
- Subqueries in WHERE
- Subqueries in HAVING
- Subqueries in ORDER BY
- Common Table Expression (CTE)
- Recursive CTE
- Check yourself





Types of subqueries

Non-correlated subqueries can be executed independently of the outer query. Such a subquery runs once, and its result is used by the outer query:

```
SELECT name, popularity
FROM tracks
WHERE popularity > (SELECT avg(popularity) FROM tracks)
ORDER BY popularity DESC
LIMIT 5;
```

RESULT

name	popularity
Peaches (feat. Daniel Caesar & Giveon)	100
Blinding Lights	96
WITHOUT YOU	94
LA NOCHE DE ANOCHE	93
DÁKITI	92



Types of subqueries

The inner query runs once and returns a single number.

Correlated subqueries reference the outer table, so they depend on the outer query. For every row of the outer query the subquery is executed again — which affects performance:

```
SELECT t.name,  
       t.popularity,  
       (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade  
FROM tracks AS t  
WHERE t.popularity > (  
    SELECT avg(t2.popularity)  
    FROM tracks AS t2  
    WHERE (EXTRACT(YEAR FROM t2.release_date)::int / 10) * 10  
          = (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10  
    )  
AND t.release_date < DATE '1940-01-01'  
ORDER BY t.popularity DESC  
LIMIT 5;
```



Types of subqueries

RESULT

name	popularity	decade
Gloomy Sunday (with Teddy Wilson & His Orchestra) - Take 1	51	1930
Sweet Home Chicago	46	1930
The Way You Look Tonight	42	1930
Cheek to Cheek	42	1930
If I Only Had A Brain - Extended Version	40	1930

"Above average **for its own decade**" — the subquery has to be re-evaluated per row, because the decade it compares against comes from the outer row.

Scalar queries are a separate group: queries returning a single value, e.g. `SELECT AVG(column) FROM table`.



Subqueries in FROM

```
SELECT *  
FROM (SELECT * FROM table) AS t1;
```

💡 Some DBMSs require subqueries in FROM to have aliases. Using aliases is a useful habit.



Subqueries in FROM

```
SELECT decade,
       count(*)           AS artists,
       round(avg(track_cnt), 1) AS avg_tracks_per_artist
FROM (
    SELECT a.id,
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,
           count(*) AS track_cnt
    FROM artists AS a
    JOIN track_artists AS ta ON ta.artist_id = a.id
    JOIN tracks AS t      ON t.id = ta.track_id
    GROUP BY 1, 2
) AS per_artist
GROUP BY decade
ORDER BY decade;
```



Subqueries in FROM

RESULT

decade	artists	avg_tracks_per_artist
1900	1	1.0
1920	273	3.8
1930	722	2.8
1940	1169	2.7
1950	1930	3.3
1960	2147	2.8
1970	2907	2.5
1980	3829	2.5
1990	6190	2.0
2000	6304	1.6
2010	8336	1.6
2020	1907	1.4



Subqueries in FROM

This is an aggregate over an aggregate — a count per artist per decade, then averaged per decade. You cannot express it in a single `GROUP BY`.





Subqueries in SELECT

Non-correlated — **must be scalar**, otherwise the DBMS cannot tell which returned value to use in a given row:

```
SELECT name,  
       popularity,  
       (SELECT round(avg(popularity), 1) FROM tracks) AS overall_avg  
FROM tracks  
ORDER BY popularity DESC  
LIMIT 5;
```

RESULT

name	popularity	overall_avg
Peaches (feat. Daniel Caesar & Giveon)	100	27.6
Blinding Lights	96	27.6
WITHOUT YOU	94	27.6
LA NOCHE DE ANOCHE	93	27.6
DÁKITI	92	27.6



Subqueries in SELECT

Correlated:

```
SELECT a.name,  
       a.followers,  
       (SELECT count(*)  
        FROM track_artists AS ta  
        WHERE ta.artist_id = a.id) AS track_cnt  
FROM artists AS a  
ORDER BY a.followers DESC  
LIMIT 5;
```

RESULT

name	followers	track_cnt
Ed Sheeran	78900234	10
Ariana Grande	61301006	14
Drake	54416812	32
Justin Bieber	44606973	37
Eminem	43747833	29



Subqueries in WHERE

Non-correlated — filtering by a constant returned by a scalar subquery, shown above.

Clauses for subqueries returning one field and several records:

IN — checks whether a value is contained in the subquery result:

```
SELECT name, popularity
FROM tracks
WHERE id IN (
    SELECT track_id
    FROM track_artists
    GROUP BY track_id
    HAVING count(*) >= 4
)
ORDER BY popularity DESC
LIMIT 5;
```

RESULT

name	popularity
3G (feat. Jon Z, Don Chezina, Chencho Corleone & Myke Towers) - Remix	83
Give Me Everything (feat. Ne-Yo, Afrojack & Nayer)	82
Hate The Other Side (with Marshmello & The Kid Laroi)	82
La Forma en Que Me Miras	78
Ayer Me Llamó Mi Ex – Remix	77



Subqueries in WHERE

EXISTS — **TRUE** if the subquery returns at least one row. It makes sense when the subquery is correlated:

```
SELECT a.name, a.followers
  FROM artists AS a
 WHERE EXISTS (
      SELECT 1
      FROM artist_related AS ar
      WHERE ar.artist_id = a.id
    )
 ORDER BY a.followers DESC
 LIMIT 5;
```

RESULT

name	followers
Ed Sheeran	78900234
Ariana Grande	61301006
Drake	54416812
Justin Bieber	44606973
Eminem	43747833



Subqueries in WHERE

`NOT EXISTS` is the natural way to ask the opposite — artists nobody is linked to:

```
SELECT count(*) AS artists_with_no_related
  FROM artists AS a
 WHERE NOT EXISTS (
     SELECT 1 FROM artist_related AS ar WHERE ar.artist_id = a.id
 );
```

RESULT

artists_with_no_related
5935



Subqueries in WHERE

ANY/SOME — **TRUE** if the condition holds for at least one value in the set. **ALL** — **TRUE** if it holds for every value:

```
SELECT name, popularity
FROM tracks
WHERE release_date >= DATE '2020-01-01'
      AND popularity > ALL (
          SELECT popularity FROM tracks WHERE release_date < DATE '1930-01-01'
      )
ORDER BY popularity DESC
LIMIT 5;
```

RESULT

name	popularity
Peaches (feat. Daniel Caesar & Giveon)	100
Blinding Lights	96
WITHOUT YOU	94
LA NOCHE DE ANOCHE	93
DÁKITI	92



Subqueries in WHERE

ANY and **SOME** are completely identical; some DBMSs support only one of the two.

💡 **IN** and **= ANY** behave differently from **NOT IN** when the subquery can return a **NULL**: **NOT IN** with a **NULL** in the list returns no rows at all. Prefer **NOT EXISTS** when the column is nullable.





Subqueries in HAVING

Only the records whose aggregates satisfy the comparison with the subquery are kept (the subquery must then return a single value):

```
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,  
       count(*) AS tracks  
FROM tracks  
GROUP BY decade  
HAVING count(*) > (SELECT count(*) / 12 FROM tracks)  
ORDER BY decade;
```

RESULT

decade	tracks
1970	6325
1980	8419
1990	11135
2000	8881
2010	10764



Subqueries in HAVING

Decades with more tracks than an even split across the twelve decades would give.





Subqueries in ORDER BY

A correlated subquery in **ORDER BY** computes a sorting value for every row:

```
SELECT a.name, a.followers
FROM artists AS a
ORDER BY (
    SELECT count(*)
    FROM track_artists AS ta
    WHERE ta.artist_id = a.id
) DESC
LIMIT 5;
```

RESULT

name	followers
Die drei ???	613060
Lata Mangeskar	2612197
TKKG Retro-Archiv	23593
Francisco Canaro	16774
Johann Sebastian Bach	3149269

💡 A non-correlated subquery in **ORDER BY** has no practical meaning — sorting by a constant does not change the order of rows.



Common Table Expression (CTE)

A CTE lets you move a subquery outside the main query. It is declared with **WITH** and can be used several times — but only within one overall query.

```
WITH cte_name AS (  
    SELECT id, name, count FROM table  
)  
SELECT *  
FROM table1  
JOIN cte_name ON table1.id = cte_name.id;
```



Common Table Expression (CTE)

Advantages:

1. **Improved readability** — a complex query is split into understandable parts.
2. **Reuse** — the CTE's result can be referenced several times without repeating the text of the subquery.

There can be several CTEs in one query: `WITH` is written once and the CTEs are listed separated by commas. A later CTE may reference an earlier one:

```
WITH artist_tracks AS (  
    SELECT a.id, a.name, t.popularity  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t        ON t.id = ta.track_id  
)  
artist_stats AS (  
    SELECT id,  
           name,  
           count(*)           AS track_cnt,  
           round(avg(popularity), 1) AS avg_popularity  
    FROM artist_tracks  
    GROUP BY id, name  
)  
SELECT name, track_cnt, avg_popularity  
FROM artist_stats  
WHERE track_cnt >= 30  
ORDER BY avg_popularity DESC  
LIMIT 10;
```



Common Table Expression (CTE)

RESULT

name	track_cnt	avg_popularity
Drake	32	63.4
Taylor Swift	32	61.1
Metallica	44	47.1
Bad Bunny	30	45.4
Luis Miguel	46	44.8
The Beatles	53	43.4
Michael Jackson	32	42.8
Justin Bieber	37	42.6
Los Tigres Del Norte	33	41.6
U2	37	41.5



Common Table Expression (CTE)

The same query written with nested subqueries would be a good deal harder to read — and this is only two levels.



Common Table Expression (CTE)

CTE vs temporary table:

- A **temporary table** is created by the user and exists until the session ends (until the connection is closed).
- A **CTE** exists only within the execution of the query; once the query has returned data, the CTE is gone.



Recursive CTE

A CTE can refer to itself. That is the only way to walk a graph or a hierarchy in SQL, and `artist_related` is exactly such a graph — "listeners also like".

```
WITH RECURSIVE seed AS (  
    SELECT id FROM artists WHERE name = 'David Bowie'  
)  
,  
reach AS (  
    SELECT id, 0 AS depth FROM seed  
    UNION ALL -- the recursive part  
    SELECT ar.related_artist_id, r.depth + 1  
    FROM reach AS r  
    JOIN artist_related AS ar ON ar.artist_id = r.id  
    WHERE r.depth < 2 -- the stop condition  
)  
SELECT min(r.depth) AS hops, a.name  
FROM reach AS r  
JOIN artists AS a ON a.id = r.id  
GROUP BY a.name  
ORDER BY hops, a.name  
LIMIT 10;
```

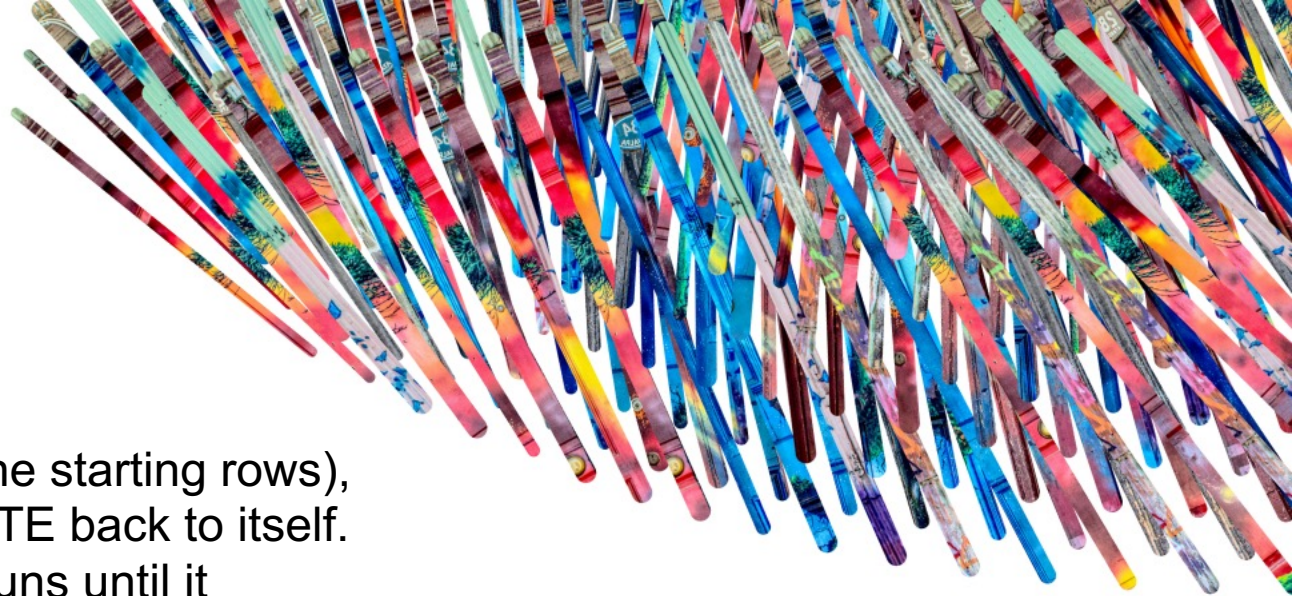
RESULT

hops	name
0	David Bowie
1	Blondie
1	Brian Eno
1	Echo & the Bunnymen
1	George Harrison
1	Iggy Pop
1	Joy Division
1	Lou Reed
1	New Order
1	Patti Smith



Recursive CTE

The structure is always the same: a **base case** (the starting rows), `UNION ALL`, and a **recursive case** that joins the CTE back to itself. Without a stop condition — here `depth < 2` — it runs until it exhausts the graph, or forever if the graph has cycles.





Check yourself

What is the difference between correlated and non-correlated subqueries? — non-correlated ones can run independently of the outer query; correlated ones depend on data from it.

How do `EXISTS`, `SOME`, `ALL` and `ANY` work? — `EXISTS` checks for at least one row; `SOME/ANY` compare against any value from the subquery; `ALL` against all of them.

What is a scalar subquery? — one that returns a single value.

What happens if a subquery in `WHERE` returns `NULL`? — the condition is not satisfied, since `NULL` is neither true nor false, so the outer query returns no rows.

What is a CTE used for? — readability and reuse.

How does a CTE differ from a temporary table? — a temporary table lives until the end of the session, a CTE only within the query.

What are the three parts of a recursive CTE? — the base case, `UNION ALL`, and the recursive case that references the CTE itself.



1.5 Joins and unions

- DISTINCT
- UNION and UNION ALL
- JOIN
- Joins on the real data
- Special types of JOIN
- Venn diagrams
- Joining and aggregating together
- Check yourself



DISTINCT

`DISTINCT` removes duplicate rows from the query result:

```
SELECT DISTINCT column1 FROM table_name;  
SELECT DISTINCT column1, column2 FROM table_name;  -- unique combinations
```

`DISTINCT` is written once, right after `SELECT`. The DBMS has to remember every unique value and compare it with the following ones, which can slow the query down.



DISTINCT

```
SELECT DISTINCT time_signature  
FROM tracks  
ORDER BY time_signature;
```

RESULT

time_signature
0
1
3
4
5



UNION and UNION ALL

`UNION` and `UNION ALL` combine the results of two or more queries **vertically**:

- `UNION` combines the results, excluding duplicate rows;
- `UNION ALL` does the same but keeps duplicates.





UNION and UNION ALL

Deduplication costs extra resources and time, so **UNION ALL** works faster.

```
SELECT <field_1>, <field_2>, <field_3>
  FROM table_1
UNION [ALL]
SELECT <field_1>, <field_2>, <field_3>
  FROM table_2;
```



UNION and UNION ALL

The restriction: the number of fields in the combined results must be the same. Field names may differ — the final names come from the first query. Data types may differ too, but combining only compatible types is recommended.

Putting the two extremes of a column side by side is a natural use:

```
(SELECT 'loudest' AS bucket, name, round(loudness::numeric, 2) AS loudness
  FROM tracks ORDER BY loudness DESC LIMIT 3)
UNION ALL
(SELECT 'quietest' AS bucket, name, round(loudness::numeric, 2)
  FROM tracks ORDER BY loudness ASC  LIMIT 3)
ORDER BY loudness DESC;
```



UNION and UNION ALL

RESULT

bucket	name	loudness
loudest	Jean-Lou	4.11
loudest	Mr. Satan	2.47
loudest	Dance on, Little Girl	2.34
quietest	Le sacre du printemps: Part 1 "Adoration of the Earth", The Wise Elder	-48.28
quietest	Le sacre du printemps (The Rite of Spring): Première partie: L'adoration de la terre (Part I: Adoration of the Earth): Le sage (The Sage)	-55.00
quietest	Pause Track	-60.00



UNION and UNION ALL

Note the parentheses: without them, `ORDER BY` and `LIMIT` would apply to the whole union rather than to each branch.

The difference between the two forms, on a query that produces duplicates:

```
SELECT count(*) AS with_union_all FROM (  
    SELECT key FROM tracks WHERE popularity > 80  
    UNION ALL  
    SELECT key FROM tracks WHERE explicit  
) AS u;
```

RESULT

with_union_all
2826



UNION and UNION ALL

```
SELECT count(*) AS with_union FROM (  
    SELECT key FROM tracks WHERE popularity > 80  
    UNION  
    SELECT key FROM tracks WHERE explicit  
) AS u;
```

RESULT

with_union
12

UNION collapsed tens of thousands of rows down to the handful of distinct keys.



JOIN

JOIN enriches data from one table with data from another — a **horizontal** combination. Every row in one table is matched against the second by some condition, and matching rows are combined.

```
SELECT a.column1, b.column2
FROM table1 a
JOIN table2 b ON a.id = b.table2_id;
```

Tables can be given aliases so that you do not have to write their names out in full; **AS** may be omitted here.



JOIN

INNER may be omitted — an inner join is performed by default. **OUTER** may be omitted in **LEFT JOIN** and **RIGHT JOIN**.

To see the four kinds clearly we need a pair of tiny tables where some rows deliberately fail to match. Here they are, built inline with **VALUES**:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity',      1),
    (2, 'Heroes',            2),
    (3, 'Blackstar',         9),    -- label 9 does not exist
    (4, 'Blonde on Blonde',  3),
    (5, 'Bedroom Demo',      NULL), -- no label at all
    (6, 'Lodger',            2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'),
    (2, 'RCA'),
    (3, 'Columbia'),
    (4, 'Sub Pop')                -- a label with no releases here
)
SELECT * FROM releases ORDER BY release_id;
```



JOIN

RESULT

release_id	title	label_id
1	Space Oddity	1
2	Heroes	2
3	Blackstar	9
4	Blonde on Blonde	3
5	Bedroom Demo	<i>NULL</i>
6	Lodger	2



JOIN

INNER JOIN returns the rows that have a match in both tables:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
INNER JOIN labels AS l ON r.label_id = l.label_id
ORDER BY r.release_id;
```

RESULT

release_id	title	label_name
1	Space Oddity	Philips
2	Heroes	RCA
4	Blonde on Blonde	Columbia
6	Lodger	RCA



JOIN

'Blackstar' (label 9, which does not exist) and 'Bedroom Demo' (**NULL**) are excluded, and so is 'Sub Pop', which has no releases.

LEFT JOIN returns all rows from the left table and the matching rows from the right one; where there is no match, **NULL**:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
LEFT JOIN labels AS l ON r.label_id = l.label_id
ORDER BY r.release_id;
```



JOIN

RESULT

release_id	title	label_name
1	Space Oddity	Philips
2	Heroes	RCA
3	Blackstar	<i>NULL</i>
4	Blonde on Blonde	Columbia
5	Bedroom Demo	<i>NULL</i>
6	Lodger	RCA



JOIN

RIGHT JOIN is the mirror image — all rows from the right table plus matches from the left. It shows which labels exist even if they have no releases:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
RIGHT JOIN labels AS l ON r.label_id = l.label_id
ORDER BY l.label_name;
```



JOIN

RESULT

release_id	title	label_name
4	Blonde on Blonde	Columbia
1	Space Oddity	Philips
2	Heroes	RCA
6	Lodger	RCA
<i>NULL</i>	<i>NULL</i>	Sub Pop



JOIN

FULL JOIN returns all rows from both tables, filling the missing values with **NULL** — every release *and* every label, matched where possible:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
FULL JOIN labels AS l ON r.label_id = l.label_id
ORDER BY r.release_id NULLS LAST;
```



JOIN

RESULT

release_id	title	label_name
1	Space Oddity	Philips
2	Heroes	RCA
3	Blackstar	<i>NULL</i>
4	Blonde on Blonde	Columbia
5	Bedroom Demo	<i>NULL</i>
6	Lodger	RCA
<i>NULL</i>	<i>NULL</i>	Sub Pop



Joins on the real data

The many-to-many between tracks and artists is what `track_artists` exists for. Reading a track with its artists means joining through it:

```
SELECT t.name AS track,  
       string_agg(a.name, ' + ' ORDER BY ta.position) AS artists,  
       t.popularity  
FROM tracks AS t  
JOIN track_artists AS ta ON ta.track_id = t.id  
JOIN artists AS a      ON a.id = ta.artist_id  
GROUP BY t.id, t.name, t.popularity  
HAVING count(*) >= 3  
ORDER BY t.popularity DESC  
LIMIT 8;
```



Joins on the real data

RESULT

track	artists	popularity
Peaches (feat. Daniel Caesar & Giveon)	Justin Bieber + Daniel Caesar + Giveon	100
La Nota	Manuel Turizo + Rauw Alejandro + Myke Towers	89
I Like It	Cardi B + Bad Bunny + J Balvin	83
3G (feat. Jon Z, Don Chezina, Chencho Corleone & Myke Towers) - Remix	Wisin + Yandel + Farruko + Jon Z + Don Chezina + Chencho Corleone + Myke Towers	83
Patience (feat. YUNGBLUD & Polo G)	KSI + YUNGBLUD + Polo G	82
Hate The Other Side (with Marshmello & The Kid Laroi)	Juice WRLD + Marshmello + The Kid LAROI + Polo G	82
Give Me Everything (feat. Ne-Yo, Afrojack & Nayer)	Pitbull + Ne-Yo + Afrojack + Nayer	82
Don't Say Goodbye (feat. Tove Lo)	Alok + Ilkay Sencan + Tove Lo	81

`string_agg` flattens the several artist rows of a track back into one string — the standard way to present a many-to-many to a human reader.



Special types of JOIN

CROSS JOIN — a join without a condition, producing the Cartesian product of two tables. Useful when every row of one table must be matched with every row of another, but dangerous: the row count grows multiplicatively.

```
WITH top3 AS (  
    SELECT name FROM tracks ORDER BY popularity DESC, name LIMIT 3  
)  
SELECT a.name AS track_1, b.name AS track_2  
FROM top3 AS a  
CROSS JOIN top3 AS b  
WHERE a.name <> b.name  
ORDER BY 1, 2;
```

RESULT

track_1	track_2
Blinding Lights	Peaches (feat. Daniel Caesar & Giveon)
Blinding Lights	WITHOUT YOU
Peaches (feat. Daniel Caesar & Giveon)	Blinding Lights
Peaches (feat. Daniel Caesar & Giveon)	WITHOUT YOU
WITHOUT YOU	Blinding Lights
WITHOUT YOU	Peaches (feat. Daniel Caesar & Giveon)



Special types of JOIN

Three rows crossed with three rows give nine, less the three where a track meets itself. Do this to `tracks` unfiltered and you ask for 3.6 billion rows.

SELF JOIN — joining a table with itself. `artist_related` links artists to artists, so reading it means joining `artists` twice:

```
SELECT a.name AS artist,  
       r.name AS also_liked  
FROM artist_related AS ar  
JOIN artists AS a ON a.id = ar.artist_id  
JOIN artists AS r ON r.id = ar.related_artist_id  
WHERE a.name = 'David Bowie'  
ORDER BY ar.position  
LIMIT 8;
```

RESULT

artist	also_liked
David Bowie	Lou Reed
David Bowie	T. Rex
David Bowie	Roxy Music
David Bowie	Iggy Pop
David Bowie	Talking Heads
David Bowie	Brian Eno
David Bowie	Queen
David Bowie	The Velvet Underground



Special types of JOIN

Anti-join — not an official join type, but a common pattern: rows in one table with no match in another. Our sample has 1,099 tracks whose credited artist was missing from the source, and this is how you find them:

```
SELECT t.name, t.release_date, t.popularity
FROM tracks AS t
LEFT JOIN track_artists AS ta ON ta.track_id = t.id
WHERE ta.track_id IS NULL
ORDER BY t.popularity DESC
LIMIT 5;
```

RESULT

name	release_date	popularity
Body Like A Back Road	2017-02-01	76
So Pretty	2021-01-20	73
Following The Sun	2020-10-02	71
Idk	2020-07-23	64
Big Fan Dulled	2017-05-07	64



Special types of JOIN

The same logic can be expressed with `NOT EXISTS`. The pattern is worth memorising: **LEFT JOIN**, then **WHERE right_table.key IS NULL**.



Venn diagrams

Venn diagrams are often used to explain joins because they show intersections and unions clearly. They do not, however, convey all the nuances of joins in relational databases — in particular what happens when values in a table repeat. A join between a table of 60,000 tracks and a table of 74,624 credits returns more than 60,000 rows, which no Venn diagram will tell you.





Joining and aggregating together

Joins and aggregation are usually used in the same query — and the combination has a trap in it. Counting tracks per artist with a `LEFT JOIN`:

```
SELECT a.name,  
       count(ta.track_id) AS track_cnt  
FROM artists AS a  
LEFT JOIN track_artists AS ta ON ta.artist_id = a.id  
GROUP BY a.name  
ORDER BY track_cnt DESC  
LIMIT 5;
```

RESULT

name	track_cnt
Die drei ???	392
Lata Mangeshkar	282
Francisco Canaro	210
TKKG Retro-Archiv	210
Johann Sebastian Bach	195



Joining and aggregating together

`count(ta.track_id)` returns 0 rather than 1 for an artist with no tracks, because `COUNT(<field>)` ignores `NULL`s. Had we written `count(*)`, every unmatched artist would have counted its own single `NULL` row as 1. This is the most common bug in a `LEFT JOIN` with aggregation:

```
count(*)           -- counts rows, including the NULL-filled unmatched ones → 1
count(ta.track_id) -- counts non-NULL values → 0. This is what you want.
```



Check yourself

How does `DISTINCT` affect performance? — it increases execution time: the DBMS has to compare and remember every unique value.

What is the difference between `UNION` and `UNION ALL`? — `UNION` removes duplicates, `UNION ALL` does not and is therefore faster.

How do you combine or join fields of different types? — cast one of them with `CAST` or `::`.

Table `t1` has `N` rows, `t2` has `M` rows. What is the maximum number of rows a join can produce? — $N \times M$, with a `CROSS JOIN`.

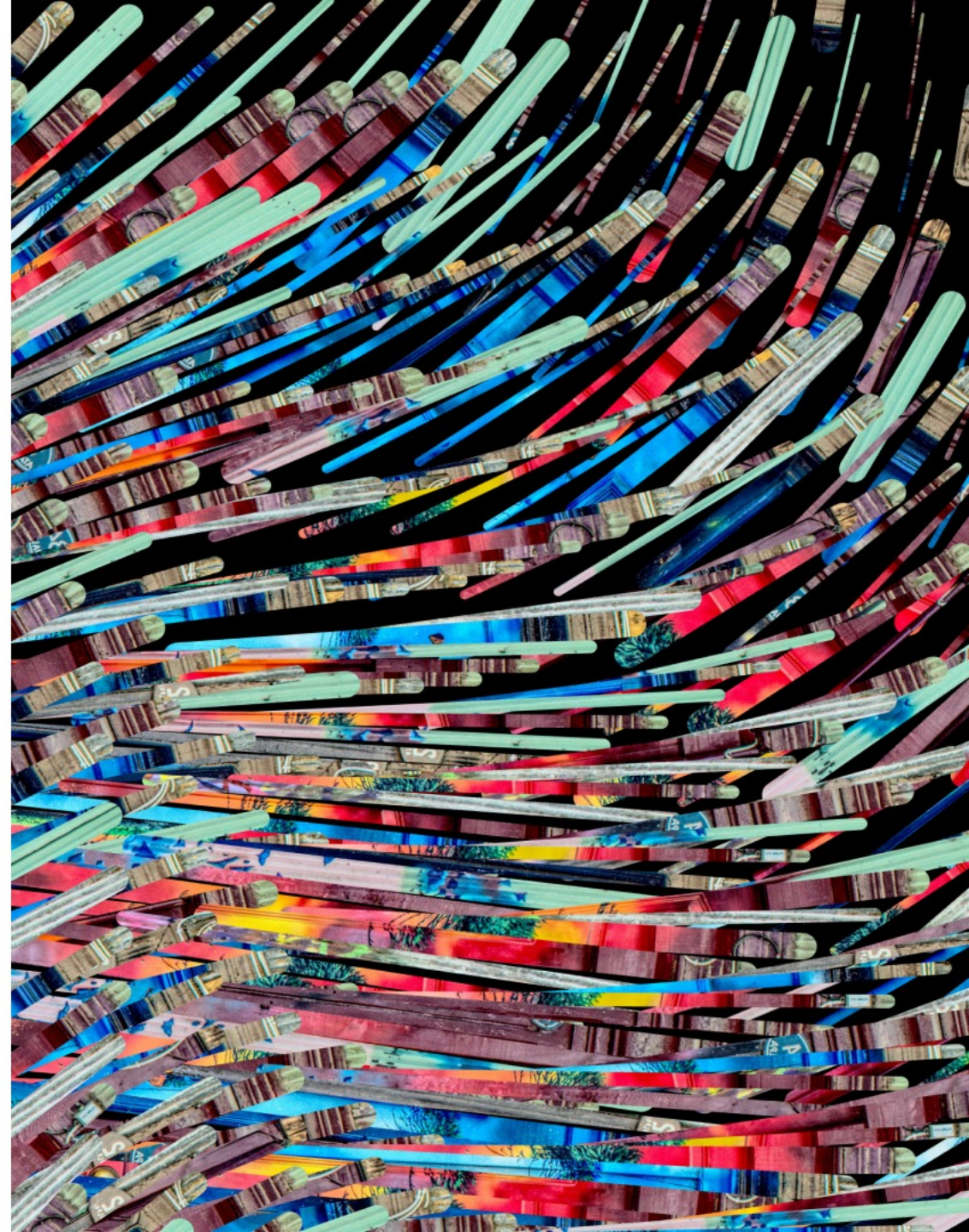
How does `JOIN` differ from `UNION`? — `JOIN` combines data horizontally, enriching rows; `UNION` combines vertically, stacking rows.

In a `LEFT JOIN` with `GROUP BY`, why use `COUNT(field)` rather than `COUNT(*)`? — unmatched rows come back with `NULL`s; `COUNT(*)` would count them as 1, `COUNT(field)` correctly gives 0.



1.6 ACID and the execution plan

- ACID
- Order of execution of clauses
- The execution plan





ACID

Anything that changes data in a database happens inside a **transaction** — a unit of work that either takes effect completely or not at all. A transaction is opened with **BEGIN** and closed with **COMMIT** (apply) or **ROLLBACK** (undo).

You can try this safely in the playground — the whole thing runs as one script:

```
BEGIN;

UPDATE artists
  SET followers = followers + 1000000
  WHERE name = 'David Bowie';

-- inside the transaction, we see the new value
SELECT name, followers FROM artists WHERE name = 'David Bowie';

ROLLBACK;

-- afterwards, as if it never happened
SELECT name, followers FROM artists WHERE name = 'David Bowie';
```

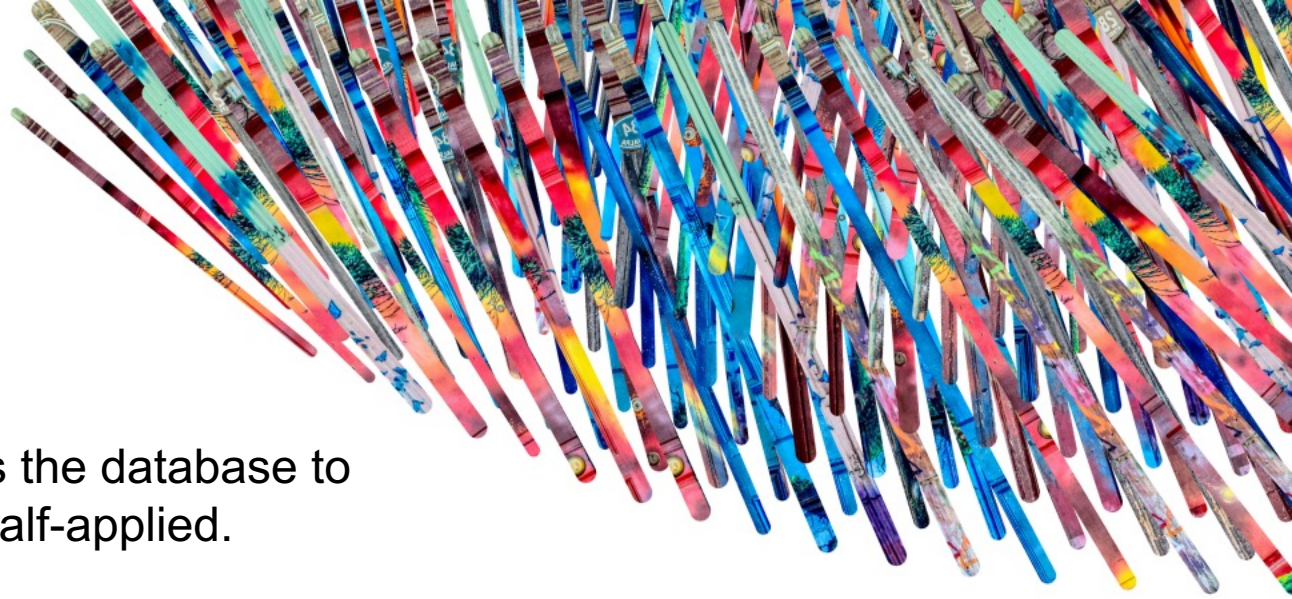
RESULT

name	followers
David Bowie	6753696



ACID

If a statement in the middle fails, **ROLLBACK** returns the database to the state it was in before **BEGIN** — nothing is left half-applied.





ACID

ACID is the set of four guarantees a transactional DBMS gives:

- **Atomicity** — a transaction is all-or-nothing. There is no state in which half of its statements have been applied.
- **Consistency** — a transaction moves the database from one valid state to another; all constraints (primary and foreign keys, `CHECK`, `NOT NULL`) hold before and after it. Our `track_artists.artist_id` has a foreign key to `artists`, so the database will simply refuse a credit for an artist that does not exist.
- **Isolation** — concurrent transactions do not see each other's intermediate results. How strictly is governed by the **isolation level**: `READ UNCOMMITTED`, `READ COMMITTED` (the PostgreSQL default), `REPEATABLE READ`, `SERIALIZABLE`. The stricter the level, the fewer the anomalies (dirty reads, non-repeatable reads, phantom reads) and the higher the cost in concurrency.
- **Durability** — once `COMMIT` returns, the data survives a crash or a power failure, because it has been written to the transaction log on disk.



ACID

Consistency is not an abstraction here. This is the foreign key doing its job:

```
INSERT INTO track_artists (track_id, artist_id, position)
VALUES ('35iwgR4jXetI318WEWsa1Q', 'no-such-artist', 0);
-- ERROR: insert or update on table "track_artists" violates foreign key constraint
```



ACID

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
-- ... the statements of the transaction ...  
COMMIT;
```

💡 Analytical (OLAP) databases often relax these guarantees deliberately in exchange for read throughput. ACID in full is the domain of transactional (OLTP) systems such as PostgreSQL.



Order of execution of clauses

We write an SQL query in one order and the DBMS executes it in another. The logical order of execution is:

1. **FROM** — the engine determines which tables the data will be taken from.
2. **ON** — the join conditions are processed; filtering happens before the join itself.
3. **JOIN** — the tables are joined on the specified conditions.
4. **WHERE** — the rows satisfying the query's conditions are filtered.
5. **GROUP BY** — the rows are grouped by the specified columns.
6. **HAVING** — the aggregated data is filtered; groups that fail the condition are cut off.
7. **SELECT** — the columns given in the query are computed and extracted.
8. **DISTINCT** — duplicates are removed from the result, if required.
9. **ORDER BY** — the data is sorted.
10. **LIMIT / OFFSET** — the rows for output are finally determined.



Order of execution of clauses

This explains several things we have already met:

- why **WHERE** cannot see an alias defined in **SELECT** (step 4 runs before step 7);
- why **ORDER BY** *can* see one (step 9 runs after step 7);
- why you cannot filter on a window function in **WHERE** (windows are computed at step 7);
- why filtering in **WHERE** is cheaper than the same filter in **HAVING**.

To simplify the picture: imagine working with two large stacks of documents. First we filter out the unnecessary pages, then combine the stacks, group them by certain features, mark the important elements, and finally order and shorten them as needed.



The execution plan

The logical order above says *what* the DBMS computes. **The execution plan** says *how* — which physical operations it will perform, in what order, and at what cost. The plan is produced by the optimizer, which analyses table statistics and chooses access methods and join algorithms.

`EXPLAIN` shows the plan without running the query:

EXPLAIN

```
SELECT a.name, count(*) AS tracks
  FROM artists AS a
 JOIN track_artists AS ta ON ta.artist_id = a.id
GROUP BY a.name
ORDER BY tracks DESC
LIMIT 10;
```

RESULT

QUERY PLAN

Limit (cost=4007.86..4007.88 rows=10 width=21)
-> Sort (cost=4007.86..4075.91 rows=27220 width=21)
Sort Key: (count(*)) DESC
-> HashAggregate (cost=3147.44..3419.64 rows=27220 width=21)
Group Key: a.name
-> Hash Join (cost=1112.16..2774.32 rows=74624 width=13)
Hash Cond: (ta.artist_id = a.id)
-> Seq Scan on track_artists ta (cost=0.00..1466.24 rows=74624 width=23)
-> Hash (cost=769.85..769.85 rows=27385 width=36)
-> Seq Scan on artists a (cost=0.00..769.85 rows=27385 width=36)



The execution plan

Read it from the inside out: the innermost, most indented nodes run first. The numbers in `cost=` are the optimizer's estimate, in arbitrary units — not milliseconds. `EXPLAIN ANALYZE` actually runs the query and adds the real timings next to the estimates, which is how you catch a bad estimate.





The execution plan

Notice the `Seq Scan` on `tracks` in plans like this one. The course database deliberately has **no index** on `tracks.popularity` or `tracks.release_date`, so that Lecture 3 has something real to fix. Reading plans, `EXPLAIN ANALYZE`, indexes and query optimization are its subject.





Summary

Filtering and sorting. `WHERE` selects rows by a condition; `AND` and `OR` combine conditions, with `AND` binding tighter — set precedence explicitly with parentheses. Text is filtered with `=`, `LIKE`, `ILIKE`, `IN` and string functions. `ORDER BY` sorts (`ASC/DESC`), `LIMIT` and `OFFSET` paginate. `NULL` means the absence of data: compare it with `IS NULL` / `IS NOT NULL`, replace it with `COALESCE`, and place it explicitly in sorts with `NULLS FIRST` / `NULLS LAST`.

Aggregate functions. `SUM`, `MIN`, `MAX`, `AVG`, `COUNT` compute a single value over a set of rows. `COUNT(*)` counts rows, `COUNT(field)` counts non-`NULL` values. `GROUP BY` applies aggregates per group, `HAVING` filters the groups afterwards, and `FILTER (WHERE ...)` conditions a single aggregate. Filter non-aggregated data in `WHERE`, before grouping.

Window functions. `<function>(<field>) OVER (PARTITION BY ... ORDER BY ... <frame>)` computes over a window while returning every input row. Three groups: aggregate (`SUM`, `AVG`, `MIN`, `MAX`, `COUNT`), offset (`LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`) and ranking (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `CUME_DIST`, `PERCENT_RANK`). Frames (`ROWS`, `RANGE`) fix the exact set of rows; `WINDOW` names a window for reuse. To filter on a window function, compute it in a CTE and filter outside.



Summary

Subqueries and CTE. A subquery is a query inside another query — non-correlated, correlated or scalar — usable in `SELECT`, `FROM`, `JOIN`, `WHERE`, `HAVING` and `ORDER BY`, with `IN`, `EXISTS`, `ANY/SOME` and `ALL`. A CTE (`WITH`) lifts a subquery out of the main query, improving readability and letting one result be reused. `WITH RECURSIVE` walks graphs and hierarchies. A CTE lives only for the duration of the query; a temporary table lives until the end of the session.

Joins and unions. `DISTINCT` removes duplicate rows. `UNION/UNION ALL` combine results vertically (`UNION` deduplicates, `UNION ALL` is faster). `JOIN` combines them horizontally: `INNER`, `LEFT`, `RIGHT`, `FULL`, `CROSS`, plus the `SELF JOIN` and anti-join patterns. A many-to-many needs a bridge table — here, `track_artists`.



Summary

ACID and execution. Transactions guarantee Atomicity, Consistency, Isolation and Durability. Clauses execute in the order `FROM` → `ON` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT/OFFSET`; `EXPLAIN` shows the physical plan the optimizer chose.

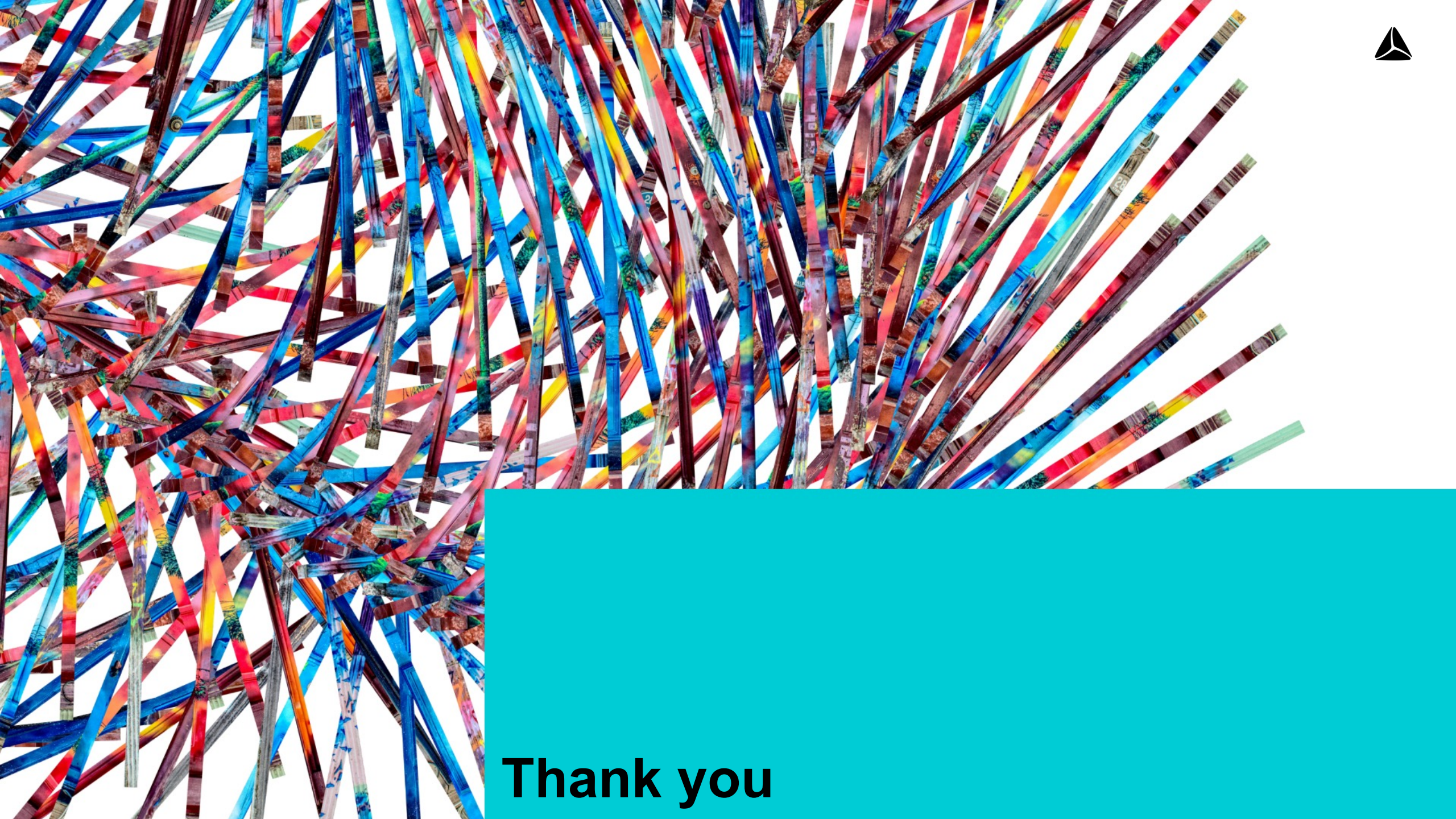
By the end of this lecture you can write an SQL query using the following clauses:

Clause	Description
<code>SELECT</code>	Queries data from one or several tables; selects specific columns and expressions
<code>DISTINCT</code>	Removes duplicate rows from the query result
<code>FROM</code>	Points to the table (or tables) the query addresses
<code>JOIN</code>	Joins rows from two or more tables on a related column: <code>INNER</code> , <code>LEFT</code> , <code>RIGHT</code> , <code>FULL</code> , <code>CROSS</code>
<code>WHERE</code>	Specifies the conditions by which rows are selected
<code>GROUP BY</code>	Groups rows with the same values in the specified columns for aggregation
<code>HAVING</code>	Filters the groups created by <code>GROUP BY</code>
<code>OVER / WINDOW</code>	Defines the window a window function is computed over
<code>WITH</code>	Declares a CTE — a named subquery used within the query; <code>WITH RECURSIVE</code> for graphs
<code>ORDER BY</code>	Orders the rows of the result (<code>ASC</code> / <code>DESC</code>)
<code>LIMIT</code> and <code>OFFSET</code>	<code>LIMIT</code> restricts the number of rows returned; <code>OFFSET</code> skips rows before returning the rest
<code>UNION</code> and <code>UNION ALL</code>	Combine the results of several <code>SELECT</code> queries; <code>UNION</code> removes duplicates



TBC Explorers

Q & A



Thank you